2.6.2.2 Metrics for LLM-based Planning Systems

To evaluate whether an LLM-Symbolic hybrid planning pipeline effectively captures user intent, evaluation is primarily conducted through human assessment of the quality of generated plans [59, 60]. Some evaluation involves analyzing the diversity and solvability of PDDL domain models produced by the LLM [14, 60]. In this thesis, the focus is placed on evaluating plan quality directly, rather than the correctness of the generated PDDL models, for two key reasons.

First, several LLM-based planners considered for comparison — such as Tree-of-Thought-based approaches [71] — do not produce explicit PDDL models, making plan-level evaluation the only consistent basis for comparison. Second, evaluating against ground-truth domain models contradicts the underlying premise that natural language task descriptions are inherently ambiguous and may admit multiple valid interpretations. As discussed in Chapter 4, enforcing a one-to-one mapping from natural language to a specific formal representation can be limiting and misleading.

More specifically, ambiguous and informal domain descriptions — especially those written in layman language — allow significant flexibility in how predicates are defined and used. Predicate-level evaluation against a fixed ground-truth domain model overlooks the overall synergy of actions in the modeling process and biases evaluation toward a single perspective. Since this thesis aims to democratize planning systems for broader user bases, predicate-level quality evaluation represents an indirect and potentially misleading measure of success.

2.6.2.3 LLMs as end-to-end planners

PlanBench [62] has been the most widely used benchmark to assess LLMs on planning and reasoning about change, drawing from domains traditionally used in the automated planning community (i.e., IPC domains). It provides a *template* to convert symbolic model information into natural language text that can be used to either train or test LLMs on plan generation. An innovation of PlanBench is its "obfuscated" planning domains, where names are replaced using misleading vocabulary. This aims to assess whether LLMs can plan based solely on the logical structure of the planning problem without any linguistic cues learned during training (see Figure 2.20). One limitation

Original Prompts vs. Obfuscated Prompts in Blocksworld Domain

Original:

```
Query:
I am playing with a set of blocks where I
need to arrange the blocks into stacks.
[ACTION DESCRIPTION]
Here are the actions that can be performed:
(a) Pick up a block
(b) Unstack a block from on top of another
block
(c) Put down a block
(d) Stack a block on top of another block

The following are the restrictions on the
actions:
(1) I can only pick up or unstack one block
at a time.
(2) I can only pick up or unstack a block if
my hand is empty.
(3) I can only pick up a block if the block
is on the table and the block is clear.  A
block is clear if the block has no other
blocks on top of it and if the block is not
picked up.

(4) I can only unstack a block from on top
of another block if the block I am unstacking
was really on top of the other block.
(5) I can only unstack a block from on top of
another block if the block I am unstacking is
clear.
(6) Once I pick up or unstack a block, I am
holding the block.

(7) I can only put down a block that I am
holding.

(8) I can only stack a block on top of
another block if I am holding the block being
stacked.
(9) I can only stack a block on top of
another block if the block onto which I am
stacking is clear.
(10) Once I put down or stack a block, my
hand becomes empty.
(11) Once you stack a block on top of a
second block, the second block is no longer
clear.
```

Obfuscated:

```
Query:
I am playing with a set of objects.

[ACTION DESCRIPTION]
Here are the actions that can be performed:
(a) Attack object
(b) Feast object from another object
(c) Succumb object

(d) Overcome object from another object

The following are the restrictions on the
actions:
(1) I can only attack or feast one object at
a time.
(2) I can only attack or feast an object if
there is harmony.
(3) I can only attack an object if the object
is a planet and the object is a province.  An
object is a province if the object does not
crave any other object and if the object is
not being attacked.

(4) I can only feast an object from on top
of another object if the object I am feasting
craves the other object.
(5) I can only feast an object from on top of
another object if the object I am feasting is
a province.
(6) Once I attack or feast an object, there
is pain with the object.

(7) I can only succumb an object if there is
pain with the object.

(8) I can only overcome an object from
another object if there is pain with the
object being overcome.
(9) I can only overcome an object from
another object if the object onto which I
am overcoming the object is a province.
(10) Once I succumb or overcome an object,
there is harmony.
(11) Once you overcome an object from a
second object, the second object is no longer
a province.
```

FIGURE 2.20: Comparison of original and obfuscated prompts in the blocksworld domain. Obfuscated vocabularies for actions are: *pick-up* → *attack*, *unstack* → *feast*, *put-down* → *succumb*, *stack* → *overcome*; Obfuscated vocabularies for objects are: *block* → *object*; Obfuscated vocabularies for predicates are: *on table* → *planet*, *clear* → *province*, *hand empty* → *harmony*, *holding* → *pain*, *on top of* → *craves*.

of the open-sourced PlanBench is that it only provides two domains, Blocks World and Logistics, which is insufficient for a comprehensive evaluation. To address this, recent studies such as Plansformer [57] and PlanGPT [58] generate additional planning domains and problem instances of varying complexity and size using PDDL generator tools [211]. Further details regarding our benchmark extension are provided in Chapter 5.

2.6.2.4 Metrics for End-to-End Plan Generation of LLMs

Several metrics are employed to evaluate the performance of LLMs as end-to-end planners. The primary focus is on *plan quality*, which is assessed using three key metrics:

Definition 2.9 (Executability of a Plan)**.** A plan is executable if it defines an action sequence $(a_0, a_1, \ldots, a_{n-1})$ with states $(s_0, s_1, \ldots, s_n)$. s_0 is the initial state and for each $i = 0, 1, \ldots, n-1$, s_{i+1} is the result of executing a_i in s_i, and the precondition of a_i must hold in s_i and s_{i+1} is the result of removing delete effects, adding add effects and applying numeric effects. The state s_n is called the final state produced by the plan and the state sequence $(s_0, s_1, \ldots, s_n)$ is called the trace of the plan. An executable plan produce a unique trace.

Definition 2.10 (Validity of a Plan)**.** A plan is valid if and only if it is executable and the goal of the problem G holds in the final state s_n produced by the plan.

These are the formal definitions written in the PDDL textbook [6], and we can see that executability is a prerequisite for a plan to be valid. This underscores the importance of including the executability metric in our evaluation of plan quality.

An additional evaluation criterion is goal satisfiability, defined as follows:

Definition 2.11 (Goal-Satisfiability of a Plan)**.** A plan is goal-satisfiable if it defines an action sequence $(a_0, a_1, \ldots, a_{n-1})$ with states $(s_0, s_1, \ldots, s_n)$. s_0 is the initial state and for each $i = 0, 1, \ldots, n-1$, s_{i+1} is the result of executing a_i in s_i, removing delete effects, adding add effects and applying numeric effects. The state s_n is called the final goal state and the goal G holds in s_n. A goal-satisfiable plan may not be executable.

Beyond the primary metrics, supplementary measures include the *optimal ratio*—the proportion of generated plans that are optimal—and *plan generation time*, which reflects the efficiency of the LLM-based planner. These measures are discussed in detail in Chapter 5.

2.7 Discussion and Chapter Summary

In this chapter, we reviewed the literature on LLMs in sequential decision-making (SDM) systems, with a focus on the two standard SDM framework — reinforcement learning

Chapter 5

Revisiting Strategies for End-to-End LLM Plan Generation

In contrast to Chapter 4, which investigated a hybrid approach where the primary planning computation was delegated to a classical planner, this chapter explores the potential of large language models (LLMs) to function as *standalone planners*. We examine whether LLMs can generate plans directly from natural language descriptions. This direct approach, if successful, could offer a simpler alternative to complex pipeline integrations and further democratize the use of planning systems.

However, the planning capabilities of LLMs remain a subject of active debate. Critics argue that strategies to boost LLMs' reasoning skills are ineffective in planning tasks [61, 62], while some studies report strong performance by simply fine-tuning models on planning-specific corpora [57, 58, 192]. This debate frames our research question (**RQ3**): Despite claims of improved reasoning skills of LLMs through various strategies, a thorough and comparative analysis of their actual effectiveness in the automated planning domain remains underexplored. Therefore, to what extent do different strategies — such as fine-tuning, Chain-of-Thought (CoT) prompting, and RL — enhance LLMs' ability to generate valid and executable plans?

To address this, we reevaluate LLMs in the context of end-to-end plan generation, with a focus on plan *executability* and *validity* (see Section 2.6.2.4). We find that merely

fine-tuning LLMs on a corpus of planning task instances does not lead to robust planning skills, as indicated by poor performance on *out-of-distribution* test sets — echoing the concerns of skeptics regarding LLMs' planning effectiveness. In contrast, strategies such as CoT prompting show incremental improvements in plan executability, suggesting some potential for enhancement. However, most strategies still fall short when it comes to improving overall plan validity. Notably, reinforcement learning, guided by our novel *Longest Contiguous Common Subsequence* (LCCS) reward, enhances both executability and validity. Collectively, our granular analysis clarifies misconceptions in the LLM-planning literature: While achieving consistently high plan validity remains challenging, our analysis shows that focused strategies yield incremental progress in plan executability — a critical prerequisite for generating valid plans. Hence, future work should aim to maintain or enhance this progress in executability while introducing mechanisms specifically designed to improve final plan validity, drawing insights from our findings[1].

This chapter builds on the paper:

Sukai, Huang, Trevor Cohn and Nir Lipovetzky. 2025. Chasing Progress, Not Perfection: Revisiting Strategies for End-to-End LLM Plan Generation. *Accepted for publication by The 35th International Conference on Automated Planning and Scheduling (ICAPS 2025) on 01-Mar-2025.*

5.1 Introduction

While latest LLMs have shown promise in areas like grade school math and code generation [256–259], their effectiveness in solving planning tasks remains a contentious issue. Numerous studies have voiced skepticism, questioning whether LLMs can truly conduct deliberative reasoning beyond statistical pattern matching [61, 64, 206]. However, the landscape is not uniformly skeptical, some studies have also made provocative claims that LLMs, when fine-tuned on paired datasets of planning problem descriptions and their corresponding plans, can generate correct plan sequences for new problems within the same domains [57, 58, 192].

[1]Codebase available at https://github.com/Sino-Huang/official-misconcept-lm-plan-gen

Existing literature on *LLMs planning* shows a clear dichotomy, likely due to limitations in evaluation methodologies on both sides of the debate, as shown below:

1. Lack of OOD Evaluation: Optimistic studies often overlook out-of-distribution (OOD) evaluation, leading to models potentially simply recalling and adapting partial plan traces from the training data, rather than demonstrating genuine sequential reasoning [260]. For instance, Shah et al. [192] evaluated their model on Sudoku puzzles of the same size as those in the training set, potentially allowing the model to reuse familiar grid patterns. Similarly, Rossetti et al. [58] evaluated their fine-tuned LLM planner on test instances generated using the same PDDL generator configurations as the training data. This lack of OOD evaluation can lead to an overestimation of the model's planning capabilities.

2. Insufficient Analysis of Failure Modes: Pessimistic studies (e.g., Valmeekam et al. [206]) typically focus solely on plan *validity*, an extremely stringent criterion that requires the entire plan to be perfectly correct. This restrictive metric fails to capture incremental improvements in planning capabilities. Moreover, these studies rarely conduct diagnostic experiments to identify specific reasoning bottlenecks, leading to a puzzling enigma in the planning community — Why do strategies that enhance reasoning capabilities in other tasks (e.g., math-solving [44]) appear ineffective in planning tasks? For instance, Stechly et al. [205] found that chain-of-thought prompting did not improve the validity rate of LLM planners. By not considering more granular metrics and failing to investigate where the strategy fails, researchers miss opportunities to understand how different strategies contribute and which aspects of reasoning need the most attention.

To address these gaps, we **reassess** various strategies for training LLMs in *end-to-end plan generation*, where the LLM itself functions as a **black-box planner** with **no** explicitly modeled *search process*. These strategies include *permutation* [261], *chain-of-thought* [44, 71], *self-correction* [257–259], and *reinforcement learning* (RL) [254]. Our evaluation extends beyond the plan *validity* metric to include plan *executability*, which, as defined in Section 2.6.2.4, refers to whether the preconditions of all actions are satisfied by the state at execution. We then employ a comprehensive suite of diagnostic experiments to identify where each strategy succeeds or fails. Moreover, OOD test sets are deliberately designed to evaluate the model's generalization capabilities in planning.

This work aims to clarify misconceptions in the LLM planning literature and provide a better understanding of the inherent planning capabilities of LLMs' *next-token prediction* mechanism. Our contributions are as follows: (1) We challenge the claim that *fine-tuning* LLMs simply on datasets containing problem contexts and reference plans acquire robust planning skills, by demonstrating their failure on OOD test sets. (2) We show that strategies like *CoT* lead to incremental improvements in plan quality by enhancing plan *executability*, even if they do not directly increase *validity* rates. (3) We show that RL with our proposed *LCCS* reward emerges as the most effective strategy. In particular, it improves plan *validity* by 7% and *executability* by 9% in longer planning problems.

5.2 Background & Related Work

The following section provides a brief recap of key background concepts relevant to this work. For details on classical planning formulations, see Section 2.4.1. Related work on using LLMs to function as *standalone planners* is reviewed in Section 2.5.2, and *next-token prediction* in LLM pretraining is discussed in Section 2.2.1.2.

Classical Planning. PDDL, the standard language for representing deterministic and fully observable planning problems, has been discussed in Section 2.4.1. A classical planning problem modeled in PDDL is a tuple $\langle \mathcal{D}, \Pi_{\mathcal{D}} \rangle$, where $\mathcal{D}$ represents the planning domain, consisting of a set of predicate symbols and action schemas; $\Pi_{\mathcal{D}}$ denotes the problem instance, which includes the objects, initial state and goal. A sequence of grounded actions $\pi = (a_0, a_1, \ldots, a_n)$ is a *valid* plan *iff* it is *executable* and the goal $\mathcal{G}$ holds in the final state after executing the plan.

Next-token prediction. We fine-tune an end-to-end LLM-planner, LM_θ, using *next-token prediction* on a text sequence $\mathbf{x} = \langle \mathcal{Z}(D), \mathcal{Z}(I), \mathcal{Z}(\pi) \rangle$, where $Z(\cdot)$ refers to the serialization of the PDDL elements into natural language (as illustrated in Figure 5.1). Let $\mathbf{x}_{<i}$ denote the first $i-1$ tokens of sequence $\mathbf{x}$, and x_i be the i-th token. Then, $\text{LM}_\theta(\hat{x}_i = x_i \mid \mathbf{x}_{<i})$ indicates the probability the model predicts the i-th token $\hat{x}_i$, to be equal to the actual token x_i, conditioned on the preceding tokens $\mathbf{x}_{<i}$. This is also known as *autoregressive modeling*. The training objective is to maximize the likelihood of the joint distribution of the training corpus $\mathcal{X}$, expressed as follows: $J(\theta) =$

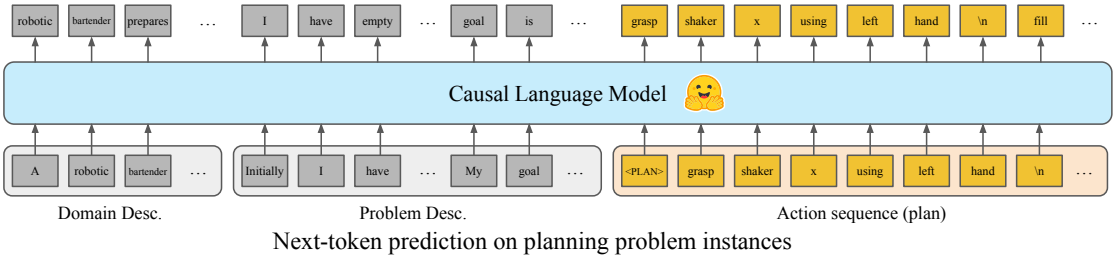


FIGURE 5.1: *Next-token prediction:* The LLM is trained to predict the next token in corpus containing domain and problem details and their plans, proceeding left-to-right.

$\max_\theta \mathbb{E}_{\mathbf{x}\sim\mathcal{X}} \left[\sum_{i=1}^{|\mathbf{x}|} \log \mathrm{LM}_\theta(\hat{x}_i = x_i \mid \mathbf{x}_{<i})\right]$. Note that this approach trains the model to predict not only tokens in the output (i.e., the generated plans) but also tokens in the input query (i.e., the domain and problem description).

Scope. We examine the planning skills of LLMs in the paradigm of **end-to-end plan generation**. Our focus excludes certain approaches, such as the LLM-Modulo framework, which relies on an external verifier to validate generated plans [61], nor the Thought of Search [197, 198], which bypasses direct long-term planning by prompting LLMs to generate search functions instead of actual plans. We focus solely on paradigms that utilize basic next-token prediction during inference. This excludes hybrid models, such as AlphaMath [193], which explicitly model *search process* via Monte Carlo Tree Search (MCTS) to derive solutions. The most closely related work to ours is the *PlanGPT* model [58]. They demonstrated that fine-tuning LLMs to predict next tokens on plan sequences, without access to the entire search trace, can effectively teach the model to generate plans for new problems.

However, our work differs in two key aspects: (1) We directly use natural language descriptions instead of PDDL snippets to train the model. This approach aligns more closely with how we expect to interact with LLMs in real-world scenarios. (2) As mentioned in Section 5.1, we conduct a more comprehensive evaluation to examine the quality of the generated plans and identify the model's reasoning bottlenecks.

Criticism on LLM Planning. Criticisms of LLMs in planning tasks stem from both theoretical analysis and empirical observations. Theoretically, LLMs are machine learning-based *probabilistic models*, and the accuracy of the models' predictions decays exponentially over the length of the sequence, a phenomenon described as "Snowballing error due to autoregressive modeling" [47]. For instance, even with a 99% correctness rate per step, the probability of a correct sequential prediction drops to about 36.6%

over 100 steps. Therefore, one has to admit that there is no guarantee of soundness in long-term planning tasks due to its probabilistic nature.

Empirically, numerous studies have demonstrated the struggles of off-the-shelf LLMs in predicting valid plans for long-term tasks [62–64]. Even advanced proprietary models like *OpenAI o1*[2], which was designed for reasoning tasks, fail on long-term planning [206]. In contrast to the pessimistic findings in the literature, we seek to better understand why LLMs often fail in planning tasks and provide insights into how to improve their planning capabilities.

5.3 Methodology & Experimental Design

5.3.1 Extended PlanBench Dataset

As mentioned in Section 2.6.2.3, PlanBench (see Figure 5.2), as introduced by Valmeekam et al. [62], has been the most widely used benchmark for evaluating planning capabilities of LLMs. It provides a *template* to convert symbolic model information into natural language text that can be used to either train or test LLMs on plan generation. An innovation of PlanBench is its "obfuscated" planning domains, where names are replaced using misleading vocabulary (see Figure 2.20 in Chapter 2). This aims to assess whether LLMs can plan based solely on the logical structure of the planning problem without any linguistic cues.

To better support both training and evaluating LLMs' planning skills, we extend the dataset in two key ways:

1. New Domains. We expand PlanBench to include additional domains from the IPC benchmarks, increasing the total from 2 to 8 domains. The new domains are Barman, Childsnack, Depots, Driverlog, Grippers and Satellite, and we design our own natural language templates to convert each domain and their problem instances into natural language descriptions for LLM training and testing on plan generation. This expansion allows for evaluation across a wider range of domains and complexities, providing a more comprehensive assessment of the model's planning capabilities.

[2]OpenAI o1 documentation: https://openai.com/o1/

PlanBench Blocksworld Example

```
Query:
[CONTEXT]
I am playing with a set of blocks where I need to arrange the blocks into stacks.
[ACTION DESCRIPTION]
Here are the actions I can do
Pick up a block
Unstack a block from on top of another block
...
I have the following restrictions on my actions:
I can only pick up or unstack one block at a time.
...
[STATEMENT]
The initial state is that, the orange block is clear,
...
My goal:  the blue block is on top of the orange block.
```

```
Response:
My plan is as follows:
[PLAN]
unstack the orange block from top of the blue block
put down the orange block
...
[PLAN END]
```

FIGURE 5.2: Each planning problem instance in PlanBench is serialized into a single text block that presents the context, action description, initial and goal states, and the plan.

2. Longer-plan Problems. A significant limitation observed in *PlanGPT* [58] was that both the training and test data shared the same distribution, a limitation discussed in Section 5.1. To address this, we have deliberately generated test instances with *longer (optimal) plan-lengths*[3], which is arguably the most straightforward way to generate OOD performance. By doing it, we prevent LLMs from easily retrieving familiar plan trace patterns from the training data, forcing them to engage in actual planning.

The updated benchmark consists of:

1. Train Set: 4000 instances per domain, with plan lengths from 3 to 16 actions. This contrasts with the previous training set, which consisted of 70,000 instances and had no restrictions on plan length.

2. Test Sets: 200 distinct instances per domain, all of which are not present in the training set, categorized into four groups: **(a) In-Distrib**: same plan length distribution as the training data. **(b) Long** (*new category*): longer plan length distribution ranging from 17 to 32 actions. **(c) Unseen** (*new category*): two novel domains not seen and not

[3]The problem's (optimal) plan-length is approximated by the plan length generated by DUAL-BFWS [191], chosen for its efficiency and ability to produce low cost plans.

trained on by the model. **(d) Obfuscated**: two domains with obfuscated predicates, actions and objects.

To prevent the model from reusing memorized plan traces and achieving artificially high performance through pattern matching, we limit the training data to 4,000 problem instances per domain. For a detailed discussion of why **smaller** datasets are preferred for both training and evaluation, see the research gap discussion in Section 2.5.2.1.

While the data size might still seem substantial, it is important to recognize that deep learning models, particularly modern LLMs, are data-hungry and are typically trained on trillions of tokens. This size represents only 5.7% of the data used in the *PlanGPT* work.

5.3.2 Strategies for Enhancing Reasoning

This section presents four strategies aiming to enhance LLMs' reasoning skills: *permutation augmentation*, *chain of thought*, *self-correction*, and *reinforcement learning*. We briefly explain the rationale and adaptation of each strategy to the planning domain.

5.3.2.1 Permutation Augmentation

Early studies have shown that data augmentations enhancing input diversity can improve a model's generalization performance [262, 263]. Recently, Allen-Zhu and Li [261] discovered that permutation augmentation — randomly rearranging sentences in a query — significantly improved *Transformer* models for question-answering tasks. They posit that exposing models to varied expressions of the same knowledge encourages them to discover the underlying structure rather than overfitting to superficial word patterns. We hypothesize a more specific rationale for why permutation augmentation works: it enhances the *self-attention* mechanism in *Transformer* during training. By permuting sentences, it ensures that when a token *attends* to others, the attended tokens do not consistently occupy the same positions. This prevents the *attention* mechanism from exploiting shortcuts based on positional or syntactic cues, which could potentially skew the model's semantic understanding. Instead, permutation encourages each token to

Permutation Augmentation in Query Content

Original:

```
Query:
...
[ACTION DESCRIPTION]
Action a
Action b
Action c
...
Action a's precon 1
Action a's precon 2
Action a's effect 1
Action a's effect 2
...
[STATEMENT]
Initial state:
1. orange block is clear
2. hand is clear
...
My goal is that:
1. red on white
2. blue on orange
...
```

+ Permutation:

```
Query:
...
[ACTION DESCRIPTION]
Action c ←
Action a ←
Action b ←
...
Action a's effect 2 ←
Action c's precon 1 ←
Action b's effect 1 ←
Action a's precon 2 ←
...
[STATEMENT]
Initial state:
1. hand is clear ←
2. orange block is clear ←
...
My goal is that:
1. blue on orange ←
2. red on white ←
...
```

FIGURE 5.3: Permutation augmentation strategy shuffles the order of action descriptions (red arrows), condition and effect descriptions (blue arrows), and atoms in initial and goal statements (green arrows). The model is expected to learn underlying semantics through more diverse data representation, avoiding overfitting to superficial patterns.

engage with semantically relevant tokens, enhancing the quality of each token's embedding. These improved embeddings enhance the model's overall ability to predict the answers.

As such, we apply it to the planning instance descriptions in the training data. Specifically, we permute the order of action descriptions and statement sentences, as illustrated in Figure 5.3. These permuted variants remain semantically equivalent, similar to how any permutation of predicates within conjunctions or disjunctions in PDDL expressions preserves their semantic meaning.

5.3.2.2 Chain of Thought (CoT)

Producing intermediate steps before directly predicting the final output, is the core idea behind CoT prompting, a strategy that has shown promise in eliciting LLMs' reasoning, as demonstrated by numerous studies [71, 264, 265]. Complementing these empirical studies, Prystawski et al. [266] provided a theoretical rationale for the effectiveness of chain-of-thought reasoning. Using a controlled Bayesian network environment, they showed that generating intermediate variables before predicting the target variable can

CoT Prompts to the Plan Response

+ Goal CoT:

```
Response:
My plan is as follows:
[PLAN]
[GOAL]
# Repeat the goal
My goal is ...
[GOAL END]
[COUNT]
# Count the remaining step to the goal
10
[COUNT END]
put down the orange block
# Next step in the plan
[GOAL]
My goal is ...
[GOAL END]
[COUNT]
9
...
[PLAN END]
```

+ State CoT:

```
Response:
My plan is as follows:
[PLAN]
[PRECON]
# Provide grounded conditions for the
action
I can only pick up a yellow block if the
yellow block is on the table and the yellow
block is clear.
...
[PRECON END]
put down the orange block
[EFFECT]
# Provide grounded effects
Once I put down a yellow block,
my hand becomes empty.
...
[EFFECT END]
...
[PLAN END]
```

FIGURE 5.4: Two types of CoT prompts are used in the plan response – *Goal CoT* and *State CoT*. *Goal CoT* prompts the agent to repeat the goal and count the remaining steps to the goal. *State CoT* prompts the agent to provide grounded conditions for the action and grounded effects. The model is expected to learn the world dynamics through these prompts.

improve a model's ability to match true conditional probabilities, particularly when relevant local variables are observed near each other in the network.

To design general CoT prompts for planning tasks, we draw inspiration from Decision Transformer [109], a RL work that uses LMs as policy models for Atari Benchmark. Both RL and planning address sequential decision problems, sharing key elements such as initial states, state transition functions, and reward/heuristic functions. In RL, Decision Transformer conditions action prediction on the provided remaining *return* and *state* information. Recognizing this common ground, we structured our CoT prompts to also include the information of the goal distance and the current state. We posit that training LLMs to predict these details before deciding on an action would help the model learn world dynamics better and thus make better decisions.

But, in our context, serializing fully-observable states directly into natural language can lead to memory explosion, potentially overwhelming the model's capacity. Thus, we instead represent states using transition information (see Figure 5.4). In fact, by estimating these details, LLMs perform the roles of both state transition and heuristic functions in classical search algorithms. An orthogonal investigation by Katz et al. [197]

Response with Erroneous Steps and Correction

```
+ Self Correction:
Response:
My plan is as follows:
# Assume the length of this reference plan is 7.
[PLAN]
1st action; 2nd action;
5th action [WRONG]; # Synthesized mistakes by selecting an action that appears later in the
plan
3rd action; 4th action;
7th action [WRONG]; # [WRONG] is special 'removal' token
5th action;
6th action; 7th action
[PLAN END]
```

FIGURE 5.5: Self-correction learning modifies plan responses to involve erroneous steps followed by their immediate corrections, with the incorrect step indicated by token `[WRONG]`.

also explored leveraging LLMs to generate state transition and heuristic function code for planning tasks, but essentially not examining the plan generation skills.

5.3.2.3 Self-Correction Learning

This strategy has shown effectiveness especially in grade school math: Kumar et al. [258] demonstrated that a multi-turn online reinforcement learning approach, which trains the model to correct its own mistakes, improves the model's accuracy. Similarly, Ye et al. [257] showed that training LLMs on data containing errors and their "immediate removal" can lead to higher accuracy compared to training on error-free data alone. Several theoretical frameworks support the effectiveness of self-correction learning, with the most intuitive being *contrastive learning*. By exposing the model to both mistakes and correct solutions, we establish a robust learning environment. In contrast, training solely on ground truth data can lead to *exposure bias*, where the model becomes overly confident in its predictions due to a limited range of correct examples. This overconfidence can hinder the model's ability to generalize effectively to unseen data [267].

This strategy has been largely overlooked in previous studies on training LLMs for plan generation. To explore its impact, we adopt an approach similar to Ye et al. [257], designing synthesized error-correction procedure in the planning steps. Specifically, we randomly select an action that appears later in the plan sequence and insert it to the current step with a special 'removal' token, as illustrated in Figure 5.5.

5.3.2.4 Reinforcement Learning (RL)

Reinforcement learning (RL) has emerged as a promising approach to enhance the reasoning capabilities of LLMs. As of early 2025, *DeepSeek R1*[4] [45, 254] demonstrated that when applying RL specifically to reasoning tasks, such as math and code generation, yields superior performance gains compared to applying RL to more general tasks. This finding, from a model rivaling state-of-the-art systems like *OpenAI o1*, highlights RL's potential to refine LLM skills in sequential reasoning. Given our focus on improving LLMs for SDM tasks, *DeepSeek R1*'s success offers valuable insight into tailoring RL strategies to enhance LLM planning skills.

However, applying RL training to LLM-based plan generation poses two challenges:

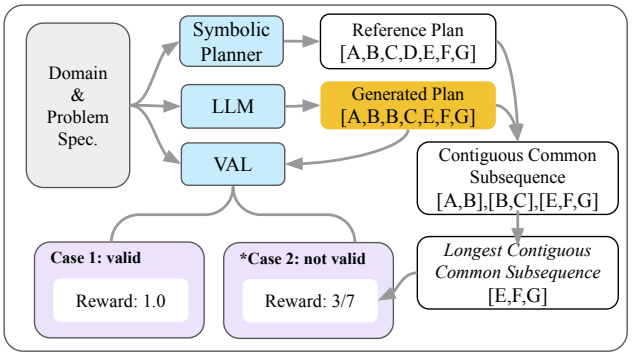
1. Typical RL frameworks for LLM are designed to attribute rewards to individual tokens as they are generated. However, in plan generation, the validity of a plan can only be determined once the entire response is complete, which creates a mismatch with the token-level optimization paradigm.

2. Using plan validity as the reward signal — assigning +1 only when plans are valid, and 0 rewards otherwise — results in very sparse feedback. Such binary and infrequent rewards often lead to poor performance in RL algorithms [268] (see Section 2.1.2.3 for a detailed discussion).

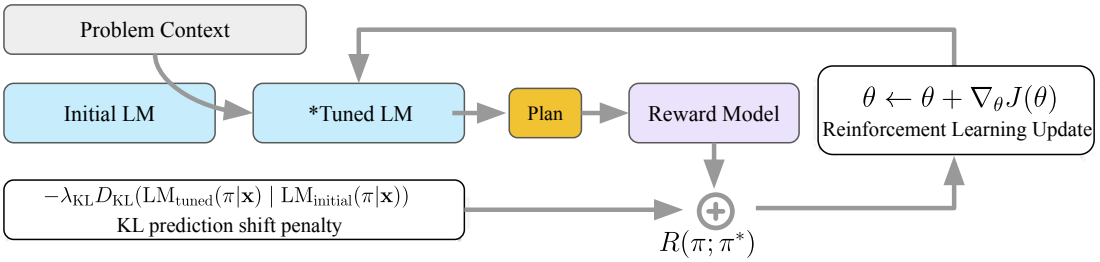
To address the first challenge, we leverage the recently proposed *rloo* framework by Ahmadian et al. [146]. *rloo* enables LLMs to optimize at the sequence level, providing delayed rewards associated with the entire output rather than individual tokens. This approach aligns better with planning tasks, where plan validity can only be assessed upon completion of the entire sequence.

To address the second challenge and create a more informative reward signal, we propose using the Longest Contiguous Common Subsequence (LCCS) for *reward shaping*. For a detailed explanation of reward shaping and its role in reinforcement learning, see Section 2.1.3. Let P_g be the generated plan and P_r be the reference plan. We define the

[4] https://huggingface.co/deepseek-ai/DeepSeek-R1



(a) Proposed LCCS Reward Model



(b) Reinforcement Learning for tuning LLM planner

FIGURE 5.6: (a) *Proposed LCCS Reward:* We use the length of LCCS as an auxiliary reward signal for RL. It provides granular feedback to fill the sparse reward gap. (See Section 5.3.2.4). (b) *Reinforcement Learning:* RL with LCCS Reward is shown to be the most effective strategy for enhancing end-to-end LLM planners among all tested strategies. In our experiments, we use PPO [**?**] as the RL algorithm. PPO constrains the policy update to a small region around the previous policy to avoid large changes that could destabilize training. In PPO, a KL divergence term is added to the objective function to penalize large deviations from the previous policy.

LCCS reward function as:

$$R(P_g, P_r) = \begin{cases} 1 & \text{if } P_g \text{ is valid} \\ \frac{|\mathrm{LCCS}(P_g, P_r)|}{|P_r|} & \text{otherwise} \end{cases} \tag{5.1}$$

When the plan is valid, we give a reward of +1. However, when the plan is invalid, a supplementary reward is also provided based on the length of the LCCS between P_g and P_r (also see Figure 5.6 part a). Note that the current reward system has an inherent bias because it relies on a single reference plan, whereas there may be multiple valid plans for a given problem. However, this limitation can be effectively addressed in future work by considering distances to multiple reference plans from top-k planners, potentially providing a more robust measure [269]. Despite this limitation, we found that LCCS serves as a good reward signal to fill the sparsity gap and provide granular feedback on the quality of the output. As such, it offers smoother gradients for the model to learn from, facilitating more effective training.

We present the LCCS-based reward shaping as one viable solution to reward sparsity in RL for LLMs in plan generation. Designing improved or alternative reward shaping approaches remains an open direction for future research.

5.4 Evaluation Results

Model We fine-tuned the open-source LLM QWEN2-7B-INSTRUCT [255] on the extended PlanBench dataset. This model was pre-trained on general text but not on code. It was also trained on a *relatively* small dataset, which reduces the risk of exposure to PlanBench or related data during pre-training. During fine-tuning (following the same procedure as in Plansformer [57]), we used 4 NVIDIA A100 GPUs on the extended PlanBench dataset (Section 5.3.1). Note that, particularly during the reinforcement learning phase, the training process occasionally encountered out-of-memory (OOM) errors — even with the batch size reduced to 1. This issue primarily stemmed from the use of CoT strategies (Section 5.3.2.2), where response sequences frequently exceeded 20,000 tokens, which is beyond what most open-source frameworks can handle. A description of the hyperparameters is provided in Table 5.1.

Evaluation Metrics In previous work, the assessment of LLM planners primarily focused on the *validity rate* of the generated plans [61, 205]. However, the validity rate alone is inadequate for differentiating between the performances of various strategies, as it is too stringent for evaluating incremental improvements in plan quality. Relying solely on this metric can obscure deeper insights into the planning process, preventing a thorough understanding of where the model faces reasoning bottlenecks and how to address them.

To address this limitation and gain a more comprehensive understanding, we employ two complementary metrics alongside validity:

Executability Rate: A plan is deemed *executable* if each action satisfies its preconditions given the state resulting from the previous action (or the initial state for the first action). This metric assesses the local consistency and step-by-step feasibility of the plan, allowing for evaluation even if the final goal is not reached. To prevent trivially empty plans from being considered executable, we only count plans with more than 3 actions in this evaluation.

TABLE 5.1: **Hyperparameters** used in fine-tuning and RL training for LLMs plan generation.

Topic	Hyperparameter	Value
Data generation	mistake rate	0.2
	Seed	1111
	train size	4000
	test size	200
Fine-tuning	model name	Qwen2-7B-Instruct
	Gradient accumulation steps	1-2
	Train epochs	2.0
	lr scheduler type	cosine
	Learning rate	1.0e-5
RL	LoRA rank	128
	LoRA alpha	256
	Gradient accumulation steps	2-4
	Seed	23
	PPO epochs	1
	Mini batches	1
	Learning rate	3.0e-6
	Total episodes	500000
	Missing EOS penalty	0.5
	LoRA dropout	0.05
Evaluation	Number of plans to generate (num_plans)	1, 3, 5
	Top-p sampling threshold (top-p)	0.93
	Token cutoff (top-k)	50

Goal Satisfiability Rate: A plan is considered *goal-satisfiable* if the cumulative application of its actions' effects results in a state where the goal condition holds, irrespective of whether the plan is fully executable. This metric evaluates the model's ability to generate a sequence that, conceptually, leads to the goal, even if the intermediate steps contain logical errors (e.g., precondition violations).

Together, these three metrics (validity, executability, and goal satisfiability) provide a more nuanced picture of LLM planning capabilities, distinguishing between local coherence and overall correctness. For further details on these metrics, please refer to Section 2.6.2.4.

In the following sections, we present our experimental results, focusing on the effectiveness of the four strategies for enhancing LLM planning capabilities. We evaluate

TABLE 5.2: Performance of the fine-tuned LLM across various test sets with no additional strategies applied. Although the LLM performs well on in-distribution, it struggles to generalize to OOD cases.

Domain	**In-Distrib.**		**Long**	
	valid. rate	*exec. rate*	*valid. rate*	*exec. rate*
BLOCKSWORLD	98.5%	98.5%	13.5%	23.5%
LOGISTICS	**100%**	**100%**	14.0%	20.5%
BARMAN	**100%**	**100%**	25.0%	43.5%
CHILDSNACK	**100%**	**100%**	66.0%	67.0%
DEPOTS	98.5%	98.5%	31.0%	37.0%
DRIVERLOG	**100%**	**100%**	31.0%	50.7%
GRIPPERS	99.0%	99.0%	50.5%	76.0%
SATELLITE	99.0%	99.2%	51.5%	53.0%
Domain	**Unseen**			
	valid. rate		*exec. rate*	
HANOI	**0%**		35%	
STORAGE	**0%**		**1%**	
Domain	**Obfuscated**			
	valid. rate		*exec. rate*	
BLOCKSWORLD	**0%**		**0%**	
LOGISTICS	**0%**		**0%**	

performance in both in-distribution and out-of-distribution test sets to provide a comprehensive assessment. To begin with, we establish a baseline by fine-tuning the LLM on the vanilla corpus. During the training phase, the model is tasked with predicting the next tokens in serialized planning instances consisting of domain descriptions, problem instance descriptions, and the associated plans. Then, in the testing phase, we prompt the model with only the domain and problem instance descriptions. The model is expected to continue the sequence by generating the plan.

5.4.1 LLMs Learn to Plan in Natural Language, but Struggle in OOD Scenarios

Previous studies have asserted the effectiveness of fine-tuning LLMs for plan generation [57, 58, 192]. We revisit this claim, examining whether the statements hold true in our extended PlanBench dataset.

5.4.1.1 In-Distribution Test: A Promising Start

Table 5.2 presents the performance of our fine-tuned LLM on the vanilla corpus. The model indeed achieved high performance across all domains in in-distribution tests, mirroring the positive result reported by *PlanGPT* [58]. Remarkably, we attain comparable performance using only 5.7% of their training data and a more complex input format expressed in natural language, suggesting not only better data efficiency but also better capability to process less structured data than previously anticipated.

5.4.1.2 Longer Planning: A Drastic Decline

The 'long' test set reveals a significant performance drop, particularly in the NP-hard BLOCKSWORLD domain [270], where the *validity rate* falls from 98.5% to 13.5%. This dramatic decline underscores the model's limitations in handling longer and more complex planning scenarios, suggesting that the planning capabilities acquired through end-to-end fine-tuning are not robust.

5.4.1.3 Childsnack: Partial Success?

In the CHILDSNACK domain, which involves preparing sandwiches for allergic and not allergic children, the model achieves the highest validity rate at 66.0% in the long' test set. However, the reasoning complexity of the tasks in this domain is not very sensitive to plan length. While the plan length expands with an increase in the number of children to feed, the order in which the children are addressed does not matter. Upon examining the generated plans, it appears that the model's major shortcoming is **not** its ability to handle food allergies, which it manages adeptly. Instead, it is the model's failure to adapt to updated world dynamics involving more children and objects, often resulting in the omission of several children from the plan. Therefore, it suggests that the model's reasoning abilities are insufficient for adapting to changes in out-of-distribution scenarios, even when the core task logic is well comprehended.

5.4.1.4 Generalization to Novel Domains: A Clear Failure

The fine-tuned model utterly failed to perform in the "unseen" and "obfuscated" test sets, unable to generate either valid or executable plans. This performance breakdown was particularly striking in the "obfuscated" test set. Here, the LLM planner often resorted to repeating irrelevant actions from domains present in the training set, neglecting the actual planning context.

Overall, our results refute the claim that fine-tuning alone enables LLMs to master complex planning. Next, we will examine whether the purported strategies can turn the tide. Examples of context prompts and model-generated responses, including those for the obfuscated BLOCKSWORLD domain and other test scenarios, are provided in Appendix C.1.

5.4.2 Majority Vote helps: *pass@k* Validity Rate

For the sake of brevity, all subsequent tables, including Tables 5.3 and 5.4, present only the average performance across all domains in the ablation study.

The *pass@k* metric is an important evaluation measure for assessing the performance of LLMs in reasoning. It provides insights into the model's ability to generate correct solutions across multiple attempts. The probabilistic nature of the generative process in LLMs results in the fact that the correct plan may not always be the most confident one. Due to this, employing a majority vote over multiple sampling outputs has become a common practice to enhance the robustness of model predictions [264]. In our context, *pass@k* metrics measure the validity rate by taking the best of k samples generated by the model. Results are shown in Table 5.3.

The results show that while multiple sampling yields limited gains on the 'unseen' and 'obfuscated' domains — reflecting the inherent challenges of generalizing to unfamiliar contexts — the *Best-of-N Sampling* consistently improves the validity rate on the 'long' test set across all strategies. Among all the ablation strategies, the vanilla (row 1) and RL (row 10) models demonstrate the most significant improvements when utilizing multiple sampling. However, the gains from this technique are notably reduced for approaches that significantly increase response length, such as those incorporating CoT

TABLE 5.3: Pass@k validity rates for different strategies across test sets. Results show consistent improvements for 'long' test sets as k increases, while 'unseen' and 'obfuscated' sets show no significant gains. Notably, the vanilla (1) and RL (10) strategies demonstrate the highest performance gains with multiple sampling

Row	Strategies					Long			Unseen			Obfuscated		
	Perm.	Goal CoT	State CoT	Self Correct	RL	pass@1	pass@3	pass@5	pass@1	pass@3	pass@5	pass@1	pass@3	pass@5
1⋆						34.8%	43.9%	49.0%	**0%**	12.2%	12.2%	**0%**	**0%**	**0%**
2	✓					35.0%	40.4%	42.9%	**0%**	**0%**	**0%**	**0%**	**0%**	**0%**
3	✓	✓				12.1%	15.6%	17.8%	5.5%	6.0%	7.0%	**0%**	**0%**	**0%**
4	✓		✓			29.5%	35.3%	37.2%	**0%**	**0%**	**0%**	**0%**	**0%**	**0%**
5	✓	✓	✓			23.8%	28.4%	34.1%	**0%**	**0%**	**0%**	**0%**	**0%**	**0%**
6	✓			✓		32.6%	40.4%	42.3%	**0%**	**0%**	**0%**	**0%**	**0%**	**0%**
7	✓	✓		✓		14.9%	21.0%	24.3%	**0%**	**0%**	**0%**	**0%**	**0%**	**0%**
8	✓		✓	✓		27.5%	31.4%	35.1%	**0%**	**0%**	**0%**	**0%**	**0%**	**0%**
9	✓	✓	✓	✓		25.9%	31.2%	36.5%	**0%**	**0%**	**0%**	**0%**	**0%**	**0%**
10⋆					✓	41.5%	49.2%	52.0%	12.5%	12.5%	12.5%	**0%**	**0%**	**0%**
11				✓	✓	36.3%	42.3%	45.0%	**0%**	**0%**	**0%**	**0%**	**0%**	**0%**

prompts (row 3 to 9), actually cannot benefit from multiple sampling. We attribute this phenomenon to the nature of autoregressive prediction in LLMs. That is, as the response length increases, the context provided by all preceding tokens becomes denser and more concrete. This richer context narrows the range of plausible continuations for the next token, leading to more focused and less diverse outputs. Consequently, the benefits of generating multiple samples are diminished for most of these strategies.

5.4.3 The Secret Help of Permutation

Our analysis reveals an intriguing finding regarding the impact of permutation augmentation. While this technique does not significantly improve the *validity rate*, it largely enhances the *executability rate* (see Table 5.4 row 2). In particular, we observe a remarkable 75.5% score in "unseen" test set, while the vanilla model only got 20.1% (row 1). This high performance suggests that permutation augmentation enables the model to effectively parse unseen contexts, which includes unseen actions, predicates and objects. This aligns with the findings of Allen-Zhu and Li [261] and underscores the importance of *data augmentation* in enhancing LLMs generalization capabilities.

5.4.4 Goal CoT: The Complexity Paradox and Overfitting Issue

Goal CoT is the only strategy that hinders planning performance among OOD cases, showing no improvement whatsoever (see Table 5.4 row 3, 5, 7, 9). We attribute the

TABLE 5.4: Ablation Study on Strategy Effectiveness in Planning. Validity rates (valid.) and the Executability rate (exec.) are analyzed. Strategies such as Permutation, CoT, and Self-Correct show no significant *validity*. improvements but enhance *executability* in 'long' and other OOD scenarios. Notably, 'Goal CoT' appears to hinder performance. We attribute this to the dual duties of generating plans and accurately estimating the heuristics of the state, which increases overall complexity and hinders the model's ability to effectively learn both aspects. RL emerges as the only strategy that enhances *validity* in OOD scenarios. Improvements of statistical significance are highlighted in green, while significant declines are highlighted in red.

Row	Strategies					In-Distrib.		Long		Unseen		Obfuscated	
	Perm.	Goal CoT	State CoT	Self Correct	RL	valid.	exec.	valid.	exec.	valid.	exec.	valid.	exec.
1						99.3%	99.8%	34.8%	42.3%	**0%**	20.1%	**0%**	**0%**
2	✓					99.5%	99.8%	35.0%	48.3%	**0%**	75.5%	**0%**	**0%**
3	✓	✓				96.8%	98.5%	12.1%	18.7%	5.5%	53.4%	**0%**	**0%**
4	✓		✓			98.9%	99.5%	29.5%	43.0%	**0%**	**100%**	**0%**	**0%**
5	✓	✓	✓			98.7%	99.0%	23.8%	39.3%	**0%**	90.8%	**0%**	**0%**
6	✓			✓		99.7%	99.9%	32.6%	50.6%	**0%**	70.9%	**0%**	**0%**
7	✓	✓		✓		97.3%	98.6%	14.9%	25.6%	**0%**	38.7%	**0%**	**0%**
8	✓		✓	✓		98.1%	99.3%	27.5%	49.4%	**0%**	94.5%	**0%**	**0%**
9	✓	✓	✓	✓		98.3%	99.1%	25.9%	30.4%	**0%**	90.6%	**0%**	**0%**
10⋆					✓	99.2%	99.6%	41.5%	51.3%	12.5%	23.0%	**0%**	**0%**
11⋆				✓	✓	99.7%	**100%**	36.3%	53.6%	**0%**	71.5%	**0%**	**0%**

failure to two factors: **1. Complexity Paradox:** Estimating the goal distance adds complexity to the planning process. Although the intention was to provide heuristic guidance, this added layer paradoxically complicates the task. The requirement to predict the steps needed to achieve a goal demands precision but also restricts the model's flexibility during planning — By fixing the total number of action steps before planning begins, the model loses the ability to dynamically adjust its plan based on the evolving conditions. **2. Poor Generalization:** The model exhibits a noticeable bias towards estimating numbers within the range of plan lengths that it has previously encountered during the training stage. This aligns with the observable limitations of LLMs in generalizing to unseen formulas and number — an issue that has been extensively highlighted by Gorceix et al. [271].

5.4.5 LLMs Recognize Mistakes But Fail to Correct Them

Despite high initial expectations for self-correction learning — stemming from its demonstrated effectiveness in solving maths, this recently proposed strategy did not improve *validity rates* (see Table 5.4 row 6-9). To gain insight into this outcome, we conducted a qualitative analysis to understand how self-correction learning affects the model's

planning behavior. The experimental setting corresponds to row 6 in Table 5.4, which combines the self-correction strategy with permutation augmentation.

Blocksworld Problem Instance Query Context

Query:

I am playing with a set of blocks where I need to arrange the blocks into stacks.

[ACTION DESCRIPTION]

Here are the actions that can be performed:

(a) Pick up a block

(b) Unstack a block from on top of another block

(c) Put down a block

(d) Stack a block on top of another block

The following are the restrictions on the actions:

...

[STATEMENT]

As initial conditions I have that

the green block is on top of the red block, the yellow block is on top of the cyan block, the blue block is clear, the orange block is on top of the magenta block, the white block is on the table, the cyan block is on top of the orange block, the blue block is on top of the black block, the black block is on top of the green block, the magenta block is on top of the white block, the red block is on top of the yellow block, the hand is empty.

My goal is to have that

the cyan block is on top of the red block, the blue block is on top of the cyan block, the green block is on top of the magenta block, the orange block is on top of the blue block, the red block is on top of the yellow block, the white block is on top of the green block, the yellow block is on top of the white block.

Response Generated By Trained LLM Planner

Response:

My plan is as follows: [PLAN]

unstack the blue block from on top of the black block

put down the blue block

unstack the blue block from on top of the yellow block WRONG

unstack the blue block from on top of the yellow block WRONG

unstack the blue block from on top of the blue block WRONG

unstack the yellow block from on top of the blue block WRONG

unstack the blue block from on top of the yellow block

put down the blue block

unstack the yellow block from on top of the blue block WRONG

unstack the yellow block from on top of the blue block WRONG

unstack the yellow block from on top of the blue block WRONG

unstack the blue block from on top of the blue block WRONG

put down the blue block

```
unstack the yellow block from on top of the blue block WRONG
unstack the yellow block from on top of the blue block WRONG
unstack the yellow block
put down the yellow block
pick up the blue block
stack the blue block on top of the yellow block
unstack the orange block from on top of the blue block WRONG
unstack the blue block from on top of the yellow block
put down the blue block
stack the orange block on top of the blue block
unstack the orange block from on top of the blue block WRONG
unstack the orange block from on top of the blue block WRONG
...
put down the orange block
unstack the blue block from on top of the yellow block
put down the blue block
pick up the orange block
stack the orange block on top of the blue block
[PLAN END]
```

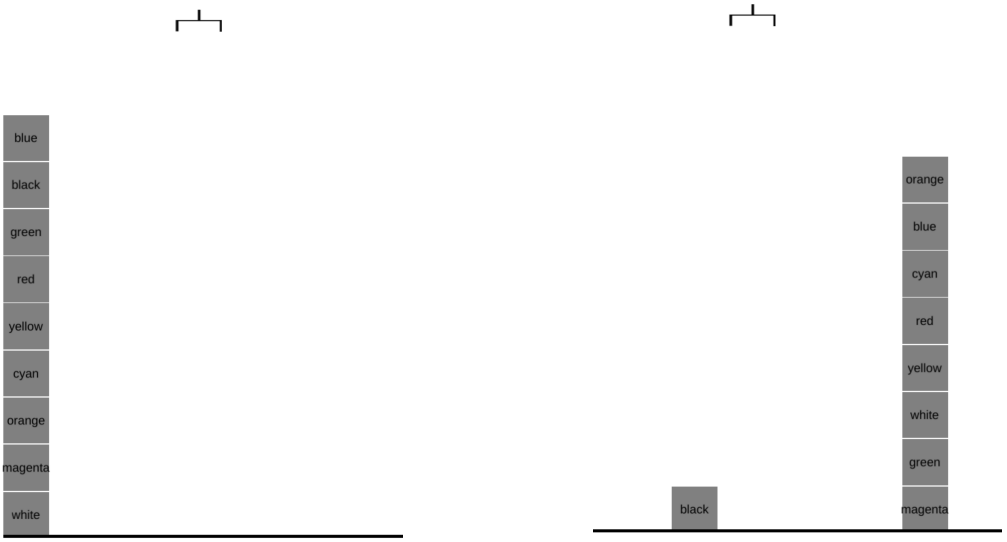
Blocksworld Problem Instance Reference Plan

```
(unstack blue black) (put-down blue) (unstack black green) (put-down black) (unstack green
red) (put-down green) (unstack red yellow) (put-down red) (unstack yellow cyan) (put-down
yellow) (unstack cyan orange) (stack cyan red) (unstack orange magenta) (stack orange
blue) (unstack magenta white) (put-down magenta) (pick-up green) (stack green magenta)
(pick-up white) (stack white green) (pick-up yellow) (stack yellow white) (unstack cyan
red) (put-down cyan) (pick-up red) (stack red yellow) (pick-up cyan) (stack cyan red)
(unstack orange blue) (put-down orange) (pick-up blue) (stack blue cyan) (pick-up orange)
(stack orange blue)
```

To better understand this issue, we visualize both the initial and goal states using the Planimation Tool[5] [272] see Figure 5.7. The plan trace shows that the model successfully identifies the first two steps — unstacking the blue block from the black block, and then placing it down. However, it fails to determine the correct next action. The visualization reveals that the next step should be to unstack the black block from the green block, but the model fails to recognize the presence of the green block in the scene. This omission is reflected in the generated plan, where no actions involve the green block, indicating a failure to accurately interpret the environment state.

As the LLM attempts to manipulate the blue block, its self-correction strategy enables it to quickly realize this approach is incorrect, leading it to generate a `[WRONG]`

[5] https://planimation-staging-181bc.web.app/gallery/BlocksWorld



Initial state of the BLOCKSWORLD problem Goal state of the BLOCKSWORLD problem

FIGURE 5.7: Visualization of the initial and goal states of the BLOCKSWORLD problem.

token. However, it struggles to recover from this failure. Notably, during retries, the model remains trapped focusing on the blue block, demonstrating a limited ability to revise its plan based on earlier mistakes. A possible remedy for this issue is to introduce a more sophisticated mechanism that provides detailed feedback on why the model's action is incorrect, enabling the model to learn from its mistakes more effectively. However, we leave this as an open question for future work.

5.4.5.1 Mistake Identification Probing Tests

The preceding analysis, along with the results in Table 5.4 (rows 6-9), shows that the self-correction strategy does not lead to improved overall performance. To better understand this outcome, we decompose the failure into two possible causes: (1) the model fails to identify its own mistakes, or (2) it detects mistakes but is unable to correct them. To investigate the first possibility, we conducted *mistake identification probing tests* to assess the model's ability to recognize incorrect action steps.

Recall that during self-correction training, we introduced a special token, `[WRONG]`, to explicitly signal an incorrect preceding action. In our probing setup, we evaluate whether the fine-tuned model, when presented with a completed action step, assigns higher probability to generating `[WRONG]` if the step is incorrect, compared to generating the standard newline token ($\backslash n$) used to proceed to the next step.

We used the 'long' test set for evaluation. Probing instances were constructed by truncating plans immediately after either a correct (positive example) or incorrect (negative example) action. For each instance, we measured the following probabilities: (1) $P(\texttt{[WRONG]} \mid \text{wrong step})$; (2) $P(\backslash n \mid \text{wrong step})$; (3) $P(\texttt{[WRONG]} \mid \text{correct step})$; (4) $P(\backslash n \mid \text{correct step})$.

Row	Precision	Recall
6	87.5	98.1
7	89.3	99.2
8	89.2	97.4
9⋆	90.5	99.2

TABLE 5.5: Probing test: LLM effectively identifies mistakes.

This setup frames mistake identification as a binary classification task. Precision and recall metrics were computed accordingly.

The results, shown in Table 5.5, demonstrate that the model can reliably detect errors. Notably, when all four strategies are applied (row 9), the model achieves high precision (90.5%) and recall (99.2%), indicating strong mistake recognition capabilities.

However, this detection ability does not translate into effective correction. This finding contrasts with prior findings from Ye et al. [257], where self-correction improved model performance in math problem solving by enabling both recognition and correction of errors. A possible explanation for this discrepancy lies in the nature of planning tasks. Mathematical errors can often be addressed locally — for instance, checking whether the variables required for calculation are correctly prepared. In contrast, replanning may require the model to look ahead multiple steps to expand the search tree and evaluate which actions lead to more plausible plans. After identifying a promising trajectory, the model can then backtrack to the point of failure to revise the plan accordingly. This essentially mirrors the forward search in traditional planning algorithms (see Section 2.4.2). This added complexity may be the cause of the model's failure to correct its mistakes.

5.4.6 State CoT Boosts Executability with a Caveat: Efficacy Limited to Short Problems

We observed that *State CoT* does not improve plan *executability* within the 'long' test set, yet it significantly enhances performance within the 'unseen' test set (e.g., 100% in row 4). Importantly, the 'unseen' test set retains the same plan length distribution as

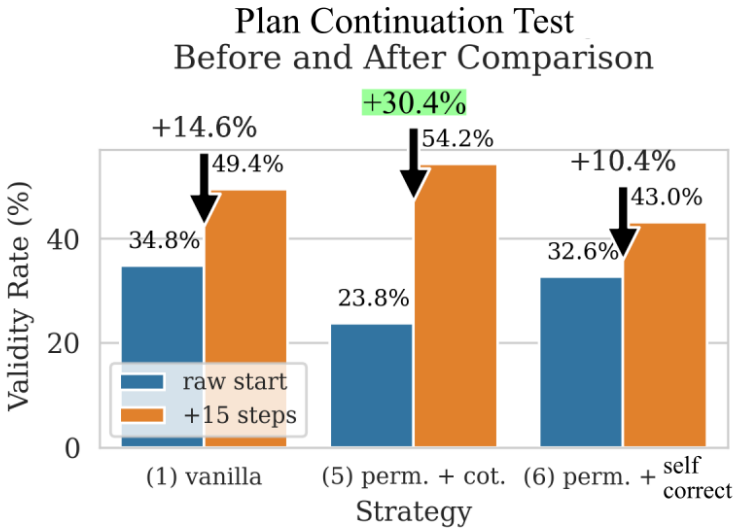


FIGURE 5.8: Validity rate of the LLM planner on the 'long' test set when provided with the first 15 actions. The model using CoT (Goal + State) showed the highest performance gain when hints were provided.

the training set. Thus, we posit that the *State CoT*'s ability to enhance the model's understanding of state transition dynamics may likely be limited to the plan length distribution it encountered during training. Consequently, we do not observe an improvement in the 'long' test set. This also rationalizes why the *State CoT* demonstrates efficacy in other reasoning tasks [44, 71], where these tasks typically require solution steps that align with the training data distribution. We shall verify this hypothesis in the next section through a 'plan continuation' experiment.

5.4.7 Familiar-Length Plan Continuation Experiments Reveal CoT's Potential

A critical question arises regarding the model's poor performance on longer problems within seen domains: Is this drop caused by distribution shift? Given that the model was trained on short-plan problem, it could have developed a bias towards shorter plans. To investigate this, we conducted a 'plan continuation' experiment, where we provided the first 15 true actions and asked the model to continue from there, ensuring that the remaining steps fell within the length distribution seen during training.

The results, as shown in Figure 5.8, reveal intriguing insights. Despite the significant hint provided, the model's performance still lags behind that of the in-distribution test set. This discrepancy is expected, as the model must infer the current world state from

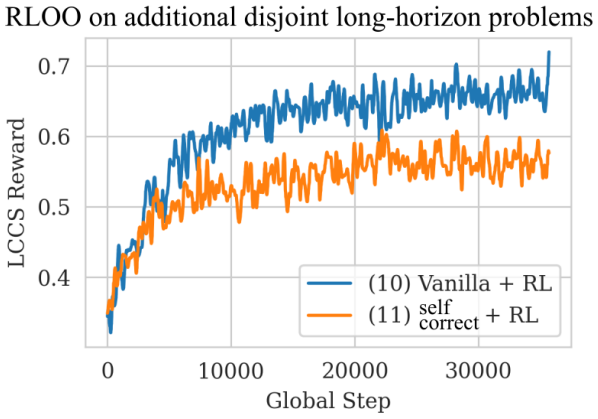


FIGURE 5.9: Reward curve of the RL training process. The LLM is further trained on 20 *disjoint* long-horizon OOD problem instances per domain. These examples are not part of the 'long' OOD test set. Despite the small training size and limited training steps, this leads to noticeable performance gains in the OOD test set.

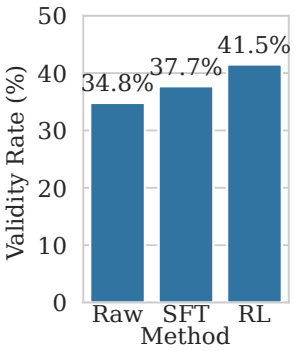


FIGURE 5.10: Comparison between RL and supervised fine-tuning (SFT), both trained on the same 20 *disjoint* long-horizon OOD instances and evaluated on the same test set. The results show that RL achieves higher validity rates.

the given actions and then continue planning to reach the goal state. Even with the initial actions provided, predicting the world state remains challenging.

Interestingly, the model employing CoT (Goal + State) demonstrates the highest performance gain when provided with the hints. Its validity rate improves dramatically from the lowest (23.8%) to the highest (54.2%) among the compared strategies. This finding suggests that when CoT operates within its "comfort zone" (i.e., in-distribution scenarios), it begins to show its effectiveness in enhancing the model's planning, supporting the hypothesis presented in Section 5.4.6. While this performance boost is encouraging, it also highlights a limitation: CoT appears to overfit to in-distribution inference. This aligns with our earlier observation that the model faces difficulty estimating the goal distance that is not within the training distribution.

5.4.8 RL Enhances Model Performance

RL notably improves performance under our end-to-end planning paradigm, especially on longer problems. In our setup, the RL phase was applied to already fine-tuned models, following standard practice in prior work (e.g., [254] and Huggingface blog[6]).

[6] https://huggingface.co/blog/trl-peft

Importantly, we chose to train the model using 20 additional long-horizon OOD problem instances per domain, which were *disjoint* from the 'long' test set, rather than continuing using in-distribution, short-horizon training data. This decision was motivated by the observation that the model had already saturated on in-distribution data after SFT (see Table 5.4). Further training on the same distribution would provide minimal reward signal differences, yielding negligible learning gradients and ineffective RL. Moreover, this small subset — just 5% of the original 4000-instance training set — is designed to address the concern that observed improvements might simply stem from an increase in data quantity, rather than from the effectiveness of the reward-driven RL optimization.

Despite the limited training data and suboptimal rewards achieved on this subset, RL boosted the validity rate on the 'long' test set from 34.8% to 41.5% (a 6.7% increase) and the executability rate from 42.3% to 53.6% (9.0%) (see Table 5.4, row 10). Interestingly, it also enabled the model to solve problems in the 'unseen' test set, achieving a 12.5% where it previously failed to generate any valid plans. The updated model does not exhibit overfitting, as indicated by the *LCCS* reward signal not reaching a perfect score of 1.0. Instead, the model has developed general planning strategies effective in unseen scenarios.

To confirm that this improvement stems from reward-driven optimization rather than a data advantage, we conducted an additional supervised fine-tuning (SFT) run using the same long-horizon OOD training subset. As shown in Figure 5.10, the RL-trained model significantly outperforms the SFT model in terms of validity rates. This finding suggests that RL fosters more comprehensive planning skills compared to supervised fine-tuning (SFT), aligning with the findings of Xiao et al. [148], Liu et al. [254].

An interesting observation is that applying RL to the vanilla model (without self-correction) resulted in faster convergence and better outcomes compared to applying RL to the self-correcting model (see Figure 5.9 and row 10 and 11 in Table 5.4). We hypothesize that the self-correction strategy, by permitting repeated attempts at actions, effectively broadens the model's state space and thus poses a greater challenge to explore a valid solution.

5.4.9 Beyond Executability: Why LLMs Still Fail to Plan?

While *executability* provides insight into the local consistency and coherence of generated plans, high executability does not guarantee that the plan achieves the desired goal — nor does it reflect the coverage of the solution space of the model, which is crucial for eventually *searching* for a valid plan. To further investigate this issue, we analyze the *goal satisfiability rate*, as defined in Definition 2.11, which offers additional insight into how well the model generates goal-directed plans. To recap, a plan is *goal-satisfiable* if the cumulative application of its actions' effects results in a state where the goal condition holds, regardless of whether the plan is fully executable. Therefore, *validity* is the conjunction of *executability* and *goal satisfiability*, meaning a plan is considered valid only if it can be executed step-by-step *and* successfully achieves the intended goal.

We compute this metric alongside executability and validity in order to understand their interaction. Interestingly, the results (Table 5.6) reveal a consistent pattern: strategies that improve *executability* do not necessarily improve — and in some cases even reduce — the *goal satisfiability rate*. This observation provides a key insight into the behavior of the model under the end-to-end plan generation paradigm.

Trade-Off Between Executability and Search Coverage The discrepancy between executability and goal satisfaction reflects a deeper trade-off between local consistency and global coverage. Autoregressive language models generate plans from left to right, prioritizing sequential coherence and local state transitions. This is especially true when strategies like *State CoT* are employed, which explicitly condition each action on the preceding state to ensure realistic transitions. While such techniques improve executability, i.e., the likelihood that each step follows from the previous one, they also constrain the model's exploration of the state space.

This results in a form of **local optimization bias**: the model becomes adept at generating locally coherent steps but overly cautious, sacrificing the broader exploration needed to discover potentially less obvious trajectories that reach the global goal.

Issues of End-to-End Plan Generation These observations highlight a fundamental limitation of the end-to-end plan generation paradigm when applied to autoregressive LLMs. Unlike traditional planning systems, which explicitly construct and traverse a

TABLE 5.6: The results show a slight decrease in the goal satisfiability rate across the different strategies. This indicates that the strategies do focus on the sequential consistency due to the nature of autoregressive models and prioritize less on the goal satisfiability.

Row	Strategies					In-Distrib.	Long	Unseen	Obfuscated
	Perm.	Goal CoT	State CoT	Self Correct	RL	goal sat.	goal sat.	goal sat.	goal sat.
1⋆						**100%**	64.5%	63.5%	**0%**
2	✓					**100%**	62.0%	25.5%	**0%**
3	✓	✓				**100%**	35.0%	27.0%	**0%**
4	✓		✓			**100%**	57.0%	**0%**	**0%**
5	✓	✓	✓			97.5%	55.5%	8.0%	**0%**
6	✓			✓		**100%**	57.5%	28.5%	**0%**
7	✓	✓		✓		96.0%	46.5%	30.5%	**0%**
8	✓		✓	✓		**100%**	56.5%	**0%**	**0%**
9	✓	✓	✓	✓		98.5%	53.5%	**0%**	**0%**
10⋆					✓	98.5%	73.5%	53.5%	**0%**
11				✓	✓	**100%**	60.0%	29.0%	**0%**

search tree, LLMs generate a single linear plan without any form of explicit backtracking, goal-checking, or search tree expansion. As a result, they lack the core capabilities provided by explicit search: they cannot systematically explore and backtrack to find alternative paths or revise their plans. This limitation highlights why higher executability does not always result in improved plan validity — because the model still lacks the ability to perform a structured search over the solution space.

Future research should therefore prioritize approaches that equip LLM-based planners with more explicit and structured search capabilities. Methods like ReAct [273] and LATS [274] exemplify promising directions in this area.

We present additional experiments in Appendix C.2, including an analysis of the quality of generated valid plans relative to optimal plans, as well as the time required for plan generation. These experiments provide supplementary insights into the performance of LLM planners.

5.5 Conclusion

We investigate the *enigma* of why strategies aimed at improving LLM reasoning often fail to achieve expected results in planning tasks. Our findings reveal that, they do enhance

the *local consistency* of the generated plans, as reflected in the improved *executability rate*. This indicates progress towards more coherent plans that better align with the state transition dynamics of the world model, despite not directly leading to valid plans.

We have limited our scope to the end-to-end plan generation paradigm, where the *search process* is not explicitly programmed. Within this context, we find that fine-tuning LLMs on a corpus of planning task instances struggles to foster robust planning skills beyond in-distribution instances. Nonetheless, our research reveals that RL stands out as the most effective strategy in this end-to-end paradigm, enhancing both the *validity* and *executability* rates on longer problems.

These findings provide a clear answer to **RQ3** — which asks: *To what extent do different strategies for enhancing reasoning skills improve LLMs' ability to generate valid and executable plans?* Our work addresses this question through a systematic comparative analysis and shows that while *self-correction* does not achieve the expected performance, strategies such as *CoT prompting* and *permutation augmentation* effectively improve the *executability* rate. Most notably, reinforcement learning with *LCCS-based reward shaping* results in enhancing both the *validity* and *executability* rates on longer problems.

This study provides a clear pathway for boosting the planning capability within the *next-token prediction* framework. Whereas previous work centered on pinpointing the shortcoming of LLMs in plan generation, we show that the **direction** of improving LLM planning lies in reward-driven RL optimization rather than in the design of complex prompting strategies. This finding opens up new avenues for future research, particularly in the realm of LLM-based planning systems.

5.5.1 Reflection and Future Work

This work operates within the scope of end-to-end plan generation using LLMs, where plans are generated in a single pass without explicitly modeling a search process.

While the findings are encouraging, they also highlight a key insight for future research: the effectiveness of planning could be further improved by introducing *explicit search*. One natural extension is to explore models that either incorporate an external search tree module (e.g., ReAct [273] and LATS [274]) or use CoT-style prompting as an

internal scratchpad to simulate search steps. These approaches offer a promising way to better balance local consistency with global goal-directed reasoning, while still staying compatible with the next-token prediction paradigm.

In addition, our findings show the value of reward-driven RL optimization in improving LLM planning capabilities. Although we adopt the *Longest Contiguous Common Subsequence (LCCS)* for reward shaping, it is just one of many possibilities. Future work may explore alternative reward signals — such as those based on goal distance (i.e., heuristic), or state-space coverage (i.e., intrinsic rewards). And the next step is to investigate how to combine these different reward signals. For instance, can we learn to adaptively weight the different types of rewards based on the current performance of the model?

Together, these directions advance LLM-based planning, making it more robust, and ultimately democratizing sequential decision-making AIs for a broader range of users.

Appendix C

Technical Appendix of Chapter 5

C.1 Problem Instances Prompts and Responses

C.1.1 Detailed Examples of Training Corpus from Driverlog Domain

In this section, we present an example problem instance and its corresponding LLM-generated responses from the DRIVERLOG domain. Additional examples can be found at: https://github.com/Sino-Huang/official-misconcept-lm-plan-gen/tree/master/data/03_primary/llm_plan_generation_dataset_qwen.

To illustrate how the focused strategies influence the responses generated by the LLM, we use Row 9 from Table 5.4 as an example, where all four strategies are applied in combination during fine-tuning.

Figure C.1 shows the problem context for this example. Subsequently, Figure C.2 presents the detailed model response, incorporating the *Goal CoT*, *State CoT*, and *Self-Correction* strategies.

Training Data Driverlog Query Content Example

```
Query:
[CONTEXT]
I have to plan how to transport objects between locations using trucks driven by drivers.
The drivers can walk between locations, board and disembark from trucks, and drive trucks
between locations.  The trucks can be loaded and unloaded with objects.
[ACTION DESCRIPTION]
Here are the actions that can be performed:
```

Load an object into a truck.
Unload an object from a truck.
A driver boards a truck.
A driver disembarks from a truck.
A driver drives a truck from one location to another location.
A driver walks from one location to another location.

The following are the restrictions on the actions:
An object can be loaded into a truck only if the object is at the same location as the truck.
Once an object is loaded into a truck, the object is not at the location but is in the truck.
An object can be unloaded from a truck only if the object is in the truck.
Once an object is unloaded from a truck, the object is not in the truck but is at the location of the truck.
A driver can board a truck only if the driver is at the same location as the truck.
A driver can board a truck only if the truck is empty.
Once a driver boards a truck, we say the driver is driving the truck.
Once a driver boards a truck, the driver is no longer at the location.
Once a driver boards a truck, the truck is no longer empty.
A driver can disembark from a truck only if the driver is driving the truck.
Once a driver disembarks from a truck, we say the driver is no longer driving the truck.
Once a driver disembarks from a truck, the driver is at the location of the truck.
Once a driver disembarks from a truck, the truck is empty.
A driver can drive a truck from one location to another location only if the driver is driving the truck and the truck is at the from-location.
A driver can drive a truck from one location to another location only if the from-location and to-location are linked.
Once a driver drives a truck from one location to another location, the truck is at the to-location and is no longer at the from-location.
A driver can walk from one location to another location only if the driver is at the from-location and the from-location and to-location are linked.
Once a driver walks from one location to another location, the driver is at the to-location and is no longer at the from-location.

[STATEMENT]
As initial conditions I have that, place 1 and place 0 are linked, driver 2 is at place 1, path 1-0 and place 0 are connected, path 1-0 and place 1 are connected, truck 1 is empty, place 1 and path 1-0 are connected, driver 1 is at place 1, package 1 is at place 1, truck 1 is at place 1, place 0 and place 1 are linked, place 0 and path 1-0 are connected.

My goal is to have that truck 1 is at place 1, driver 2 is at place 0, package 1 is at place 0.

FIGURE C.1: The LLM planner is trained on these lengthy natural language queries that describe the context, action description, initial and goal states in the Driverlog domain. It's important to recognize that the next-token prediction training of LLMs during fine-tuning isn't limited to predicting just the response tokens. In fact, the model is trained to predict all tokens, including those in the query context, similar to the pretraining process. This aspect is often overlooked in the planning community, where people may assume that the model is only trained to generate the plan response.

Training Data Response Section in Driverlog Domain When Applied CoT and Self-Correction

```
Response:
My plan is as follows:
[PLAN]
<GOAL> My goal is to have that truck 1 is at place 1, driver 2 is at place 0, package 1 is
at place 0. <COUNT> 6 </COUNT> <PRECON> A driver 1 can board a truck 1 only if the driver
1 is at the same place 1 as the truck 1. A driver 1 can board a truck 1 only if the truck
1 is empty. </PRECON> driver 1 boards truck 1 at place 1 <EFFECT> Once a driver 1 boards
a truck 1, the truck 1 is no longer empty. Once a driver 1 boards a truck 1, we say the
driver 1 is driving the truck 1. Once a driver 1 boards a truck 1, the driver 1 is no
longer at the place 1. </EFFECT>
<GOAL> My goal is to have that truck 1 is at place 1, driver 2 is at place 0, package 1 is
at place 0. <COUNT> 5 </COUNT> <PRECON> An package 1 can be unloaded from a truck 1 only
if the package 1 is in the truck 1. </PRECON> unload package 1 from truck 1 at place 0
<EFFECT> Once an package 1 is unloaded from a truck 1, the package 1 is not in the truck 1
but is at the place 0 of the truck 1. </EFFECT> WRONG
<GOAL> My goal is to have that truck 1 is at place 1, driver 2 is at place 0, package 1 is
at place 0. <COUNT> 5 </COUNT> <PRECON> An package 1 can be loaded into a truck 1 only if
the package 1 is at the same place 1 as the truck 1. </PRECON> load package 1 into truck
1 at place 1 <EFFECT> Once an package 1 is loaded into a truck 1, the package 1 is not at
the place 1 but is in the truck 1. </EFFECT>
<GOAL> My goal is to have that truck 1 is at place 1, driver 2 is at place 0, package 1
is at place 0. <COUNT> 4 </COUNT> <PRECON> A driver 2 can walk from place 1 to path 1-0
only if the driver 2 is at the place 1 and the place 1 and path 1-0 are linked. </PRECON>
driver 2 walks from place 1 to path 1-0 <EFFECT> Once a driver 2 walks from place 1 to
path 1-0, the driver 2 is at the path 1-0 and is no longer at the place 1. </EFFECT>
<GOAL> My goal is to have that truck 1 is at place 1, driver 2 is at place 0, package 1 is
at place 0. <COUNT> 1 </COUNT> <PRECON> A driver 2 can walk from path 1-0 to place 0 only
if the driver 2 is at the path 1-0 and the path 1-0 and place 0 are linked. </PRECON>
driver 2 walks from path 1-0 to place 0 <EFFECT> Once a driver 2 walks from path 1-0 to
place 0, the driver 2 is at the place 0 and is no longer at the path 1-0. </EFFECT> WRONG
<GOAL> My goal is to have that truck 1 is at place 1, driver 2 is at place 0, package 1 is
at place 0. <COUNT> 3 </COUNT> <PRECON> A driver 2 can walk from path 1-0 to place 0 only
if the driver 2 is at the path 1-0 and the path 1-0 and place 0 are linked. </PRECON>
driver 2 walks from path 1-0 to place 0 <EFFECT> Once a driver 2 walks from path 1-0 to
place 0, the driver 2 is at the place 0 and is no longer at the path 1-0. </EFFECT>
<GOAL> My goal is to have that truck 1 is at place 1, driver 2 is at place 0, package 1 is
at place 0. <COUNT> 2 </COUNT> <PRECON> A driver 1 can drive a truck 1 from place 1 to
```

```
place 0 only if the driver 1 is driving the truck 1 and the truck 1 is at the place 1.  A
driver 1 can drive a truck 1 from place 1 to place 0 only if the place 1 and place 0 are
linked.  </PRECON> driver 1 drives truck 1 from place 1 to place 0 <EFFECT> Once a driver
1 drives a truck 1 from place 1 to place 0, the truck 1 is at the place 0 and is no longer
at the place 1.  </EFFECT>
<GOAL> My goal is to have that truck 1 is at place 1, driver 2 is at place 0, package 1 is
at place 0.  <COUNT> 1 </COUNT> <PRECON> An package 1 can be unloaded from a truck 1 only
if the package 1 is in the truck 1.  </PRECON> unload package 1 from truck 1 at place 0
<EFFECT> Once an package 1 is unloaded from a truck 1, the package 1 is not in the truck 1
but is at the place 0 of the truck 1.  </EFFECT>
<GOAL> My goal is to have that truck 1 is at place 1, driver 2 is at place 0, package 1 is
at place 0.  <COUNT> 0 </COUNT> <PRECON> A driver 1 can drive a truck 1 from place 0 to
place 1 only if the driver 1 is driving the truck 1 and the truck 1 is at the place 0.  A
driver 1 can drive a truck 1 from place 0 to place 1 only if the place 0 and place 1 are
linked.  </PRECON> driver 1 drives truck 1 from place 0 to place 1 <EFFECT> Once a driver
1 drives a truck 1 from place 0 to place 1, the truck 1 is at the place 1 and is no longer
at the place 0.  </EFFECT>
[PLAN END]
```

FIGURE C.2: The *Goal CoT* contains two components, one the the <GOAL> tag that prompts the agent to repeat the goal, and the other is the <COUNT> tag that prompts the agent to count the remaining steps to the goal. The *State CoT* contains two components, one is the <PRECON> tag that prompts the agent to provide grounded conditions for the action, and the other is the <EFFECT> tag that prompts the agent to provide grounded effects. Both the preconditions and effects are specifically tied to the actual objects present in the problem instance. In this example, there are two incorrect steps in the plan, which are marked as *WRONG*. The first incorrect step borrowed the second to last step from the reference plan. The second step provides an incorrect value for the remaining steps to the goal.

C.1.1.1 Additinoal Detail for the Error Correction Data Synthesization

In our current setting, the erroneous action step is generated by randomly selecting an action from later in the reference plan sequence and inserting it to the current position, followed by the `[WRONG]` token, indicating that the current step is incorrect and should be retried. This method introduces an inductive bias that encourages the model not to skip steps. That is, we want the model to not taking shortcuts to the states that are closer to the goal state, but to strictly obey the state transition dynamics. In our experiments, we also explored an alternative approach that just select irrelevant actions from the action space. This alternative approach will generate the erroneous step by modifying the current correct action step based on two strategies: (1) either keeping the action constant while changing to other valid objects or (2) keeping the object constant while changing to other valid actions. However, this alternative method produced worse

results than our current approach. It suggests that the inductive bias introduced by our current method is beneficial for the model to learn the state transition dynamics.

C.1.2 Obfuscated Prompts: Blocksworld

Obfuscated domain replaces the name of the predicates, actions and objects with unfamiliar vocabulary. We have presented the obfuscated prompts in Figure 2.20. The LLM planner had no exposure to the obfuscated domain during training. Our results showed that achieved 0% validity and executability rates on the obfuscated test set. Upon examining the generated plans, we discovered an intriguing pattern: the model tends to ignore the obfuscated domain context and instead produced actions from the original Blocksworld domain. This behavior persisted even when we provided the first action step as a hint, as illustrated in Figure C.3.

Response Example for Obfuscated Blocksworld Problem

```
Response:
My plan is as follows:
[PLAN]
# Provide the first action step as a hint
feast object a from object b
# The LLM planner starts from here
put down object 1 # object 1 comes from nowhere
unstack the blue block from the orange block
put down the blue block
unstack the orange block from the red block
put down the orange block
unstack the red block from the blue block
put down the red block
unstack the blue block from the yellow block
put down the blue block
unstack the yellow block from the red block
put down the yellow block
unstack the orange block from the blue block
put down the orange block
pick up the blue block
stack the blue block on top of the yellow block
pick up the orange block
stack the orange block on top of the blue block
unstack the white block from the orange block
put down the white block
unstack the orange block from on top of the blue block
put down the orange block
unstack the blue block from on top of the yellow block
```

```
put down the blue block
pick up the orange block
stack the orange block on top of the blue block
pick up the white block
stack the white block on top of the orange block
[PLAN END]
```

FIGURE C.3: We observed an interesting pattern in the responses generated by the LLM planner for obfuscated blocksworld problems. The planner tends to neglect the obfuscated domain context and continue to generate the actions based on the original domain context. This behavior remains even when we provide the first action step as a hint.

An optimistic interpretation of this behavior is that the model is capable of linking the obfuscated domain back to the original domain. But eventually the model treat the obfuscated domain as an outlier and revert back to the original domain context, demonstrating that LLMs are highly sensitive to the specific vocabulary used in their training data, and struggle to generalize its planning capabilities to scenarios that deviate from its training distribution.

Two noteworthy observations emerge from the generated plan which we often call it as *hallucination*:

1. The model creates actions like "put down object 1," where "object 1" doesn't exist in either the obfuscated or original problem descriptions.

2. Interestingly, the actions generated for the original domain context often form an executable plan sequence. However, there's no one-to-one correspondence between the objects in the original and obfuscated versions.

C.2 Additional Experiments

we present a set of supplementary experiments that offer additional insights into the behavior and performance of LLM planners. While these evaluations are not the primary focus of our study, they serve to contextualize the limitations and strengths of current LLM-based planning systems.

C.2.1 Optimality Ratio Results

The optimality ratio is a metric that quantifies the proportion of valid plans that are also optimal with respect to a reference planner. It provides insight into how efficiently a model can reach the goal using minimal or most efficient steps. While our study does not

aim to optimize for this metric, reporting it offers a useful perspective on the quality of the generated plans. Table C.1 shows that in-distribution domains generally yield higher optimality, while longer and more complex tasks pose a greater challenge.

TABLE C.1: Optimality ratio of the LLM planner across different test sets.

Test Class	Domain	Optimal Ratio
In-Distrib.	barman	40.8%
	blocksworld	96.9%
	childsnack	50.0%
	depots	58.7%
	driverlog	86.3%
	grippers	54.5%
	logistics	65.8%
	satellite	84.6%
Long	barman	0.0%
	blocksworld	11.1%
	childsnack	0.0%
	depots	16.0%
	driverlog	10.1%
	grippers	6.1%
	logistics	36.0%
	satellite	15.0%
Obfuscated	blocksworld	NaN
	logistics	NaN
Unseen	hanoi	100%
	storage	NaN

C.2.2 Plan Validity of a SOTA LLM (*Gemini-2.0*)

The following experiment evaluates a top-performing closed-source LLM (GEMINI-2.0-FLASH-THINKING-EXP), currently ranked first on the Chatbot Arena leaderboard (as of December 2024). Despite its strong general capabilities, the model struggled with most planning domains under few-shot prompting when the problems require longer plan lengths to solve. These results support our view that plan validity remains a significant challenge even for state-of-the-art LLMs. Nevertheless, evaluating proprietary models remains outside the primary scope of this work due to limited transparency and replicability.

TABLE C.2: Valid plan ratio of a state-of-the-art proprietary model (GEMINI-2.0-FLASH-THINKING-EXP) across long-horizon tasks.

Test Class	Domain	Valid Plan Ratio
Long	barman	0/20
	blocksworld	2/20
	childsnack	6/20
	depots	1/20
	driverlog	0/20
	grippers	4/20
	logistics	0/20
	satellite	0/20

C.2.3 *Qwen2-7B-Instruct* Without Fine-Tuning

We also evaluated the performance of the QWEN2-7B-INSTRUCT model without any fine-tuning. The model was prompted with a few-shot example to ensure it understands the task and the output format. The results are shown in Table C.3. These results confirm previous findings [61, 62, 206] that off-the-shelf LLMs are ineffective for planning tasks without domain-specific fine-tuning.

TABLE C.3: Valid plan generation results of QWEN2-7B-INSTRUCT without fine-tuning.

Test Class	Valid Plan Ratio
In-Distrib.	1.1%
Long	0.0%
Obfuscated	0.0%
Unseen	0.0%

C.2.4 Time Required to Generate a Valid Plan

In our experiments, we utilized the DUAL-BFWS-FFPARSER planner within the *Planning as a Service* (PaaS) framework[1], setting a timeout of 0.5 seconds. We observed that all plans, with lengths up to 32 steps, were generated within this timeframe. It is very efficient for modern planners to find satisfactory plans.

The time difference of the the inference speed of Plansformer and other models is mainly due to (1) the model size and (2) GPU computational power. Our *Qwen-2* model, with 7 billion parameters, averages 3.36 seconds per plan, which is considered relatively

[1] https://github.com/AI-Planning/planning-as-a-service

small by today's standards. While Plansformer achieved a much faster inference time of approximately 0.1 seconds per instance, this was due to their use of a significantly smaller CodeT5 model with only 0.22 billion parameters.

The Google's *Gemini* tend to be the slowest, and their exact parameter sizes are not publicly available. Note that this SOTA model exhibits slower performance due to its deliberate generation of an extra *thinking process*. While this enhances reasoning capabilities, it also requires the model to generate more tokens, thus longer processing time.

Overall, these timings offer useful context on the computational characteristics of different approaches to plan generation. The results are summarized in Table C.4.

TABLE C.4: Average time required to generate a valid plan across different models.

Model	Avg. Time (sec)
DUAL-BFWS-FFPARSER (Planning as a service)	< 0.5
QWEN-2 7B FINETUNED (w/ A100 GPU)	3.36
GEMINI-2.0-FLASH-THINKING-EXP (Proprietary)	8.24

Bibliography

[1] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin A. Riedmiller, Andreas Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nat.*, 518(7540): 529–533, 2015.

[2] Hongyuan Mei, Mohit Bansal, and Matthew R. Walter. Listen, attend, and walk: Neural mapping of navigational instructions to action sequences. In *AAAI*, pages 2772–2778. AAAI Press, 2016.

[3] Felix Hill, Olivier Tieleman, Tamara von Glehn, Nathaniel Wong, Hamza Merzic, and Stephen Clark. Grounded language learning fast and slow. In *ICLR*. OpenReview.net, 2021.

[4] Blai Bonet and Hector Geffner. Planning as heuristic search. *Artif. Intell.*, 129 (1-2):5–33, 2001.

[5] Silvia Richter and Matthias Westphal. The LAMA planner: Guiding cost-based anytime planning with landmarks. *J. Artif. Intell. Res.*, 39:127–177, 2010.

[6] Patrik Haslum, Nir Lipovetzky, Daniele Magazzeni, and Christian Muise. *An Introduction to the Planning Domain Definition Language*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool Publishers, 2019.

[7] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *ICML*, pages 278–287. Morgan Kaufmann, 1999.

[8] Andrew Y. Ng and Stuart Russell. Algorithms for inverse reinforcement learning. In *ICML*, pages 663–670. Morgan Kaufmann, 2000.

[9] Jack Koch, Lauro Langosco, Jacob Pfau, James Le, and Lee Sharkey. Objective robustness in deep reinforcement learning. *CoRR*, abs/2105.14111, 2021.

[10] Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. In *ICLR*. OpenReview.net, 2022.

[11] Lauro Langosco di Langosco, Jack Koch, Lee D. Sharkey, Jacob Pfau, and David Krueger. Goal misgeneralization in deep reinforcement learning. In *ICML*, volume 162 of *Proceedings of Machine Learning Research*, pages 12004–12019. PMLR, 2022.

[12] Rodrigo Toro Icarte, Toryn Q. Klassen, Richard Anthony Valenzano, and Sheila A. McIlraith. Reward machines: Exploiting reward function structure in reinforcement learning. *J. Artif. Intell. Res.*, 73:173–208, 2022.

[13] Prasoon Goyal, Scott Niekum, and Raymond J. Mooney. Using natural language for reward shaping in reinforcement learning. In *IJCAI*, pages 2385–2391. ijcai.org, 2019.

[14] Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. LLM+P: empowering large language models with optimal planning proficiency. *CoRR*, abs/2304.11477, 2023.

[15] Jason Wei, Maarten Bosma, Vincent Y. Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V. Le. Finetuned language models are zero-shot learners. In *ICLR*. OpenReview.net, 2022.

[16] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21:140:1–140:67, 2020.

[17] Linxi Fan, Guanzhi Wang, Yunfan Jiang, Ajay Mandlekar, Yuncong Yang, Haoyi Zhu, Andrew Tang, De-An Huang, Yuke Zhu, and Anima Anandkumar. Minedojo: Building open-ended embodied agents with internet-scale knowledge. In *NeurIPS*, 2022.

[18] Yuqing Du, Olivia Watkins, Zihan Wang, Cédric Colas, Trevor Darrell, Pieter Abbeel, Abhishek Gupta, and Jacob Andreas. Guiding pretraining in reinforcement learning with large language models. In *ICML*, volume 202 of *Proceedings of Machine Learning Research*, pages 8657–8677. PMLR, 2023.

[19] Duo Zheng, Shijia Huang, Lin Zhao, Yiwu Zhong, and Liwei Wang. Towards learning a generalist model for embodied navigation. In *CVPR*, pages 13624–13634. IEEE, 2024.

[20] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

[21] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *NeurIPS*, 2020.

[22] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models. *CoRR*, abs/2302.13971, 2023.

[23] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, Jeff Klingner, Apurva Shah, Melvin Johnson, Xiaobing Liu, Lukasz Kaiser, Stephan Gouws, Yoshikiyo Kato, Taku Kudo, Hideto Kazawa, Keith Stevens, George Kurian, Nishant Patil, Wei Wang, Cliff Young, Jason Smith, Jason Riesa, Alex Rudnick, Oriol Vinyals, Greg Corrado, Macduff Hughes, and Jeffrey Dean. Google's neural machine translation system: Bridging the gap between human and machine translation. *CoRR*, abs/1609.08144, 2016.

[24] Cong Duy Vu Hoang, Philipp Koehn, Gholamreza Haffari, and Trevor Cohn. Iterative back-translation for neural machine translation. In *NMT@ACL*, pages 18–24. Association for Computational Linguistics, 2018.

[25] Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay V. Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, Yuhuai Wu, Behnam Neyshabur, Guy Gur-Ari, and Vedant Misra. Solving quantitative reasoning problems with language models. In *NeurIPS*, 2022.

[26] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *CoRR*, abs/2402.03300, 2024.

[27] Tian Ye, Zicheng Xu, Yuanzhi Li, and Zeyuan Allen-Zhu. Physics of language models: Part 2.1, grade-school math and the hidden reasoning process. *CoRR*, abs/2407.20311, 2024.

[28] Joshua Robinson and David Wingate. Leveraging large language models for multiple choice question answering. In *ICLR*. OpenReview.net, 2023.

[29] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. In *NeurIPS*, 2022.

[30] Jaehun Jung, Lianhui Qin, Sean Welleck, Faeze Brahman, Chandra Bhagavatula, Ronan Le Bras, and Yejin Choi. Maieutic prompting: Logically consistent reasoning with recursive explanations. In *EMNLP*, pages 1266–1279. Association for Computational Linguistics, 2022.

[31] Zonglin Yang, Li Dong, Xinya Du, Hao Cheng, Erik Cambria, Xiaodong Liu, Jianfeng Gao, and Furu Wei. Language models as inductive reasoners. In *EACL (1)*, pages 209–225. Association for Computational Linguistics, 2024.

[32] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NIPS*, pages 5998–6008, 2017.

[33] Jiahui Yu, Yuanzhong Xu, Jing Yu Koh, Thang Luong, Gunjan Baid, Zirui Wang, Vijay Vasudevan, Alexander Ku, Yinfei Yang, Burcu Karagol Ayan, Ben Hutchinson, Wei Han, Zarana Parekh, Xin Li, Han Zhang, Jason Baldridge, and

Yonghui Wu. Scaling autoregressive models for content-rich text-to-image generation. *Trans. Mach. Learn. Res.*, 2022, 2022.

[34] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *ICCV*, pages 4172–4182. IEEE, 2023.

[35] Keyu Tian, Yi Jiang, Zehuan Yuan, Bingyue Peng, and Liwei Wang. Visual autoregressive modeling: Scalable image generation via next-scale prediction. In *NeurIPS*, 2024.

[36] Xinlong Wang, Xiaosong Zhang, Zhengxiong Luo, Quan Sun, Yufeng Cui, Jinsheng Wang, Fan Zhang, Yueze Wang, Zhen Li, Qiying Yu, Yingli Zhao, Yulong Ao, Xuebin Min, Tao Li, Boya Wu, Bo Zhao, Bowen Zhang, Liangdong Wang, Guang Liu, Zheqi He, Xi Yang, Jingjing Liu, Yonghua Lin, Tiejun Huang, and Zhongyuan Wang. Emu3: Next-token prediction is all you need. *CoRR*, abs/2409.18869, 2024.

[37] Chunting Zhou, Lili Yu, Arun Babu, Kushal Tirumala, Michihiro Yasunaga, Leonid Shamis, Jacob Kahn, Xuezhe Ma, Luke Zettlemoyer, and Omer Levy. Transfusion: Predict the next token and diffuse images with one multi-modal model. *CoRR*, abs/2408.11039, 2024.

[38] Xiaokang Chen, Zhiyu Wu, Xingchao Liu, Zizheng Pan, Wen Liu, Zhenda Xie, Xingkai Yu, and Chong Ruan. Janus-pro: Unified multimodal understanding and generation with data and model scaling, 2025. URL https://arxiv.org/abs/2501.17811.

[39] Vishal Pallagani, Bharath C. Muppasani, Kaushik Roy, Francesco Fabiano, Andrea Loreggia, Keerthiram Murugesan, Biplav Srivastava, Francesca Rossi, Lior Horesh, and Amit P. Sheth. On the prospects of incorporating large language models (llms) in automated planning and scheduling (APS). In *ICAPS*, pages 432–444. AAAI Press, 2024.

[40] Gail Weiss, Yoav Goldberg, and Eran Yahav. Thinking like transformers. In *ICML*, volume 139 of *Proceedings of Machine Learning Research*, pages 11080–11090. PMLR, 2021.

[41] Hattie Zhou, Arwen Bradley, Etai Littwin, Noam Razin, Omid Saremi, Joshua M. Susskind, Samy Bengio, and Preetum Nakkiran. What algorithms can transformers learn? A study in length generalization. In *ICLR*. OpenReview.net, 2024.

[42] Arvid Frydenlund. The mystery of the pathological path-star task for language models. In *EMNLP*, pages 12493–12516. Association for Computational Linguistics, 2024.

[43] Xinyun Chen, Ryan A. Chi, Xuezhi Wang, and Denny Zhou. Premise order matters in reasoning with large language models. In *ICML*. OpenReview.net, 2024.

[44] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.

[45] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL https://arxiv.org/abs/2501.12948.

[46] Yuwei Fu, Haichao Zhang, Di Wu, Wei Xu, and Benoit Boulet. Furl: Visual-language models as fuzzy rewards for reinforcement learning. In *ICML*. OpenReview.net, 2024.

[47] Gregor Bachmann and Vaishnavh Nagarajan. The pitfalls of next-token prediction. In *ICML*. OpenReview.net, 2024.

[48] Tairan Fu, Javier Conde, Gonzalo Martínez, María Grandury, and Pedro Reviriego. Multiple choice questions: Reasoning makes large language models (llms) more self-confident even when they are wrong, 2025. URL https://arxiv.org/abs/2501.09775.

[49] Hengyuan Hu, Denis Yarats, Qucheng Gong, Yuandong Tian, and Mike Lewis. Hierarchical decision making by generating and following natural language instructions. In *NeurIPS*, pages 10025–10034, 2019.

[50] Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. ALFRED: A benchmark for interpreting grounded instructions for everyday tasks. In *CVPR*, pages 10737–10746. Computer Vision Foundation / IEEE, 2020.

[51] Thomas Carta, Clément Romac, Thomas Wolf, Sylvain Lamprier, Olivier Sigaud, and Pierre-Yves Oudeyer. Grounding large language models in interactive environments with online reinforcement learning. In *ICML*, volume 202 of *Proceedings of Machine Learning Research*, pages 3676–3713. PMLR, 2023.

[52] Zihao Wang, Shaofei Cai, Guanzhou Chen, Anji Liu, Xiaojian Ma, and Yitao Liang. Describe, explain, plan and select: Interactive planning with llms enables open-world multi-task agents. In *NeurIPS*, 2023.

[53] Russell Kaplan et al. Beating atari with natural language guided reinforcement learning. *ArXiv*, abs/1704.05539, 2017. URL https://api.semanticscholar.org/CorpusID:6022828.

[54] Prasoon Goyal, Scott Niekum, and Raymond J. Mooney. Pixl2r: Guiding reinforcement learning using natural language by mapping pixels to rewards. In *CoRL*, volume 155 of *Proceedings of Machine Learning Research*, pages 485–497. PMLR, 2020.

[55] Juan Rocamonde, Victoriano Montesinos, Elvis Nava, Ethan Perez, and David Lindner. Vision-language models are zero-shot reward models for reinforcement learning. In *ICLR*. OpenReview.net, 2024.

[56] Yufei Wang, Zhanyi Sun, Jesse Zhang, Zhou Xian, Erdem Biyik, David Held, and Zackory Erickson. RL-VLM-F: reinforcement learning from vision language foundation model feedback. In *ICML*. OpenReview.net, 2024.

[57] Vishal Pallagani, Bharath Muppasani, Biplav Srivastava, Francesca Rossi, Lior Horesh, Keerthiram Murugesan, Andrea Loreggia, Francesco Fabiano, Rony Joseph, and Yathin Kethepalli. Plansformer tool: Demonstrating generation of symbolic plans using transformers. In *IJCAI*, pages 7158–7162. ijcai.org, 2023.

[58] Nicholas Rossetti, Massimiliano Tummolo, Alfonso Emilio Gerevini, Luca Putelli, Ivan Serina, Mattia Chiari, and Matteo Olivato. Learning general policies for planning through GPT models. In *ICAPS*, pages 500–508. AAAI Press, 2024.

[59] Tom Silver, Soham Dan, Kavitha Srinivas, Joshua B. Tenenbaum, Leslie Pack Kaelbling, and Michael Katz. Generalized planning in PDDL domains with pretrained large language models. In *AAAI*, pages 20256–20264. AAAI Press, 2024.

[60] Lin Guan, Karthik Valmeekam, Sarath Sreedharan, and Subbarao Kambhampati. Leveraging pre-trained large language models to construct and utilize world models for model-based task planning. In *NeurIPS*, 2023.

[61] Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. Position: Llms can't plan, but can help planning in llm-modulo frameworks. In *ICML*. OpenReview.net, 2024.

[62] Karthik Valmeekam, Matthew Marquez, Alberto Olmo Hernandez, Sarath Sreedharan, and Subbarao Kambhampati. Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change. In *NeurIPS*, 2023.

[63] Karthik Valmeekam, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati. On the planning abilities of large language models - A critical investigation. In *NeurIPS*, 2023.

[64] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *CoRR*, abs/2311.05232, 2023.

[65] Jelena Luketina, Nantas Nardelli, Gregory Farquhar, Jakob N. Foerster, Jacob Andreas, Edward Grefenstette, Shimon Whiteson, and Tim Rocktäschel. A survey of reinforcement learning informed by natural language. In *IJCAI*, pages 6309–6317. ijcai.org, 2019.

[66] Brian Ichter, Anthony Brohan, Yevgen Chebotar, Chelsea Finn, Karol Hausman, Alexander Herzog, Daniel Ho, Julian Ibarz, Alex Irpan, Eric Jang, Ryan Julian, Dmitry Kalashnikov, Sergey Levine, Yao Lu, Carolina Parada, Kanishka Rao, Pierre Sermanet, Alexander Toshev, Vincent Vanhoucke, Fei Xia, Ted Xiao, Peng Xu, Mengyuan Yan, Noah Brown, Michael Ahn, Omar Cortes, Nicolas Sievers, Clayton Tan, Sichun Xu, Diego Reyes, Jarek Rettinghouse, Jornell Quiambao, Peter Pastor, Linda Luu, Kuang-Huei Lee, Yuheng Kuang, Sally Jesmonth, Nikhil J. Joshi, Kyle Jeffrey, Rosario Jauregui Ruano, Jasmine Hsu, Keerthana Gopalakrishnan, Byron David, Andy Zeng, and Chuyuan Kelly Fu. Do as I can, not as I say:

Grounding language in robotic affordances. In *CoRL*, volume 205 of *Proceedings of Machine Learning Research*, pages 287–318. PMLR, 2022.

[67] Ishika Singh, Gargi Singh, and Ashutosh Modi. Pre-trained language models as prior knowledge for playing text-based games. In *AAMAS*, pages 1729–1731. International Foundation for Autonomous Agents and Multiagent Systems (IFAAMAS), 2022.

[68] Shuang Li, Xavier Puig, Chris Paxton, Yilun Du, Clinton Wang, Linxi Fan, Tao Chen, De-An Huang, Ekin Akyürek, Anima Anandkumar, Jacob Andreas, Igor Mordatch, Antonio Torralba, and Yuke Zhu. Pre-trained language models for interactive decision-making. In *NeurIPS*, 2022.

[69] Jessy Lin, Yuqing Du, Olivia Watkins, Danijar Hafner, Pieter Abbeel, Dan Klein, and Anca D. Dragan. Learning to model the world with language. In *ICML*. OpenReview.net, 2024.

[70] Andrew Szot, Max Schwarzer, Harsh Agrawal, Bogdan Mazoure, Rin Metcalf, Walter Talbott, Natalie Mackraz, R. Devon Hjelm, and Alexander T. Toshev. Large language models as generalizable policies for embodied tasks. In *ICLR*. OpenReview.net, 2024.

[71] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *NeurIPS*, 2023.

[72] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach (4th Edition)*. Pearson, 2020.

[73] Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.

[74] Richard S Sutton. Learning to predict by the methods of temporal differences. *Machine learning*, 3:9–44, 1988.

[75] Lingheng Meng, Rob Gorbet, and Dana Kulić. Memory-based deep reinforcement learning for pomdps. In *2021 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pages 5619–5626. IEEE, 2021.

[76] Dibya Ghosh, Jad Rahme, Aviral Kumar, Amy Zhang, Ryan P Adams, and Sergey Levine. Why generalization in rl is difficult: Epistemic pomdps and implicit partial observability. *Advances in neural information processing systems*, 34:25502–25515, 2021.

[77] Yongbo Chen and Hanna Kurniawati. Pomdp planning for object search in partially unknown environment. *Advances in Neural Information Processing Systems*, 36:53146–53157, 2023.

[78] Xianping. Guo and O Hernández-Lerma. *Continuous-time markov decision processes: theory and applications*. Springer-Verlag, 2009.

[79] Hector Geffner and Blai Bonet. *A concise introduction to models and methods for automated planning*. Morgan & Claypool Publishers, 2013.

[80] Richard Bellman. A markovian decision process. *Journal of mathematics and mechanics*, pages 679–684, 1957.

[81] Lin Xiao. On the convergence rates of policy gradient methods. *Journal of Machine Learning Research*, 23(282):1–36, 2022.

[82] Alekh Agarwal, Sham M. Kakade, J. Lee, and Gaurav Mahajan. On the theory of policy gradient methods: Optimality, approximation, and distribution shift. *J. Mach. Learn. Res.*, 22:98:1–98:76, 2019.

[83] Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *J. Artif. Intell. Res.*, 47:253–279, 2013.

[84] Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune. First return, then explore. *Nat.*, 590(7847):580–586, 2021.

[85] Candace C Croney and Sarah T Boysen. Acquisition of a joystick-operated video task by pigs (sus scrofa). *Frontiers in Psychology*, 12:631755, 2021.

[86] Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine learning*, 8: 279–292, 1992.

[87] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *NIPS*, pages 1106–1114, 2012.

[88] Richard S. Sutton, David A. McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *NIPS*, pages 1057–1063. The MIT Press, 1999.

[89] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *ICLR (Poster)*, 2016.

[90] John Schulman, Sergey Levine, Pieter Abbeel, Michael I. Jordan, and Philipp Moritz. Trust region policy optimization. In *ICML*, volume 37 of *JMLR Workshop and Conference Proceedings*, pages 1889–1897. JMLR.org, 2015.

[91] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017.

[92] David Abel, Will Dabney, Anna Harutyunyan, Mark K Ho, Michael Littman, Doina Precup, and Satinder Singh. On the expressivity of markov reward. *Advances in Neural Information Processing Systems*, 34:7799–7812, 2021.

[93] Jack Clark. Faulty reward functions in the wild. https://openai.com/index/faulty-reward-functions/, 2016. Accessed: 2024-08-06.

[94] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard S. Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nat. Mach. Intell.*, 2(11):665–673, 2020.

[95] Nick Collins. Lecture notes: Lecture 24. https://www.clear.rice.edu/comp200/02spring/Lecture-notes/lec24.txt, 2002. Accessed on March 17, 2025.

[96] Yuting Tang, Xin-Qiang Cai, Yao-Xiang Ding, Qiyu Wu, Guoqing Liu, and Masashi Sugiyama. Reinforcement learning from bagged reward. In *ICML 2024 Workshop: Aligning Reinforcement Learning Experimentalists and Theorists*, 2024. URL https://openreview.net/forum?id=JbF2Ng4Dvs.

[97] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *AISTATS*, volume 9 of *JMLR Proceedings*, pages 249–256. JMLR.org, 2010.

[98] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. In *International Conference on Learning Representations*, 2018.

[99] Tianjun Zhang, Huazhe Xu, Xiaolong Wang, Yi Wu, Kurt Keutzer, Joseph E. Gonzalez, and Yuandong Tian. Noveld: A simple yet effective exploration criterion. In *Neural Information Processing Systems*, 2021.

[100] Shanchuan Wan, Yujin Tang, Yingtao Tian, and Tomoyuki Kaneko. Deir: Efficient and robust exploration through discriminative-model-based episodic intrinsic rewards. In *International Joint Conference on Artificial Intelligence*, 2023.

[101] Danijar Hafner, Timothy P. Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *ICML*, volume 97 of *Proceedings of Machine Learning Research*, pages 2555–2565. PMLR, 2019.

[102] Danijar Hafner, Timothy P. Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *ICLR*. OpenReview.net, 2020.

[103] Danijar Hafner, Timothy P. Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. URL https://openreview.net/forum?id=0oabwyZbOu.

[104] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models, 2024. URL https://arxiv.org/abs/2301.04104.

[105] Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. Reasoning with language model is planning with world model. In *EMNLP*, pages 8154–8173. Association for Computational Linguistics, 2023.

[106] Steven D. Morad, Ryan Kortvelesy, Matteo Bettini, Stephan Liwicki, and Amanda Prorok. Popgym: Benchmarking partially observable reinforcement learning. In *ICLR*. OpenReview.net, 2023.

[107] Clare Chen, Vincent Ying, and Dillon Laird. Deep q-learning with recurrent neural networks. *stanford cs229 course report*, 4:3, 2016.

[108] Marco Pleines, Matthias Pallasch, Frank Zimmer, and Mike Preuss. Generalization, mayhems and limits in recurrent proximal policy optimization, 2022. URL https://arxiv.org/abs/2205.11104.

[109] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. In *NeurIPS*, pages 15084–15097, 2021.

[110] Eric Wiewiora, Garrison W. Cottrell, and Charles Elkan. Principled methods for advising reinforcement learning agents. In *ICML*, pages 792–799. AAAI Press, 2003.

[111] Ching-An Cheng, Andrey Kolobov, and Adith Swaminathan. Heuristic-guided reinforcement learning. In *NeurIPS*, pages 13550–13563, 2021.

[112] Andrew G. Barto, Steven J. Bradtke, and Satinder P. Singh. Learning to act using real-time dynamic programming. *Artif. Intell.*, 72(1-2):81–138, 1995.

[113] Blai Bonet and Hector Geffner. Planning with incomplete information as heuristic search in belief space. In *AIPS*, pages 52–61. AAAI, 2000.

[114] Blai Bonet and Hector Geffner. Labeled RTDP: improving the convergence of real-time dynamic programming. In *ICAPS*, pages 12–21. AAAI, 2003.

[115] Simon Ståhlberg, Blai Bonet, and Hector Geffner. Learning general policies with policy gradient methods. In *KR*, pages 647–657, 2023.

[116] Zeyuan Allen-Zhu and Xiaoli Xu. DOGE: Reforming AI Conferences and Towards a Future Civilization of Fairness and Justice. *SSRN Electronic Journal*, feb 2025. doi: 10.2139/ssrn.5127931. https://ssrn.com/abstract=5127931.

[117] Harris Chan, Volodymyr Mnih, Feryal Behbahani, Michael Laskin, Luyu Wang, Fabio Pardo, Maxime Gazeau, Himanshu Sahni, Dan Horgan, Kate Baumli, Yannick Schroecker, Stephen Spencer, Richie Steigerwald, John Quan, Gheorghe Comanici, Sebastian Flennerhag, Alexander Neitz, Lei M Zhang, Tom Schaul, Satinder Singh, Clare Lyle, Tim Rocktäschel, Jack Parker-Holder, and Kristian Holsheimer. Vision-language models as a source of rewards. In *Second Agent Learning in Open-Endedness Workshop*, 2023. URL https://openreview.net/forum?id=Xw1hVTWxxQ.

[118] Silviu Pitis. Failure modes of learning reward models for llms and other sequence models. In *ICML 2023 Workshop The Many Facets of Preference-Based Learning*, 2023.

[119] Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul F. Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *CoRR*, abs/1909.08593, 2019.

[120] Nathan Lambert and Roberto Calandra. The alignment ceiling: Objective mismatch in reinforcement learning from human feedback, 2024. URL https://arxiv.org/abs/2311.00168.

[121] Jacob Eisenstein, Chirag Nagpal, Alekh Agarwal, Ahmad Beirami, Alexander Nicholas D'Amour, Krishnamurthy Dj Dvijotham, Adam Fisch, Katherine A Heller, Stephen Robert Pfohl, Deepak Ramachandran, Peter Shaw, and Jonathan Berant. Helping or herding? reward model ensembles mitigate but do not eliminate reward hacking. In *First Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=5u1GpUkKtG.

[122] Houda Nait El Barj and Theophile Sautory. Reinforcement learning from llm feedback to counteract goal misgeneralization, 2024. URL https://arxiv.org/abs/2401.07181.

[123] Pierre Sermanet, Tianli Ding, Jeffrey Zhao, Fei Xia, Debidatta Dwibedi, Keerthana Gopalakrishnan, Christine Chan, Gabriel Dulac-Arnold, Sharath Maddineni, Nikhil J. Joshi, Pete Florence, Wei Han, Robert Baruch, Yao Lu, Suvir Mirchandani, Peng Xu, Pannag Sanketi, Karol Hausman, Izhak Shafran,

Brian Ichter, and Yuan Cao. Robovqa: Multimodal long-horizon reasoning for robotics. In *ICRA*, pages 645–652. IEEE, 2024.

[124] Matthew J. Hausknecht and Peter Stone. Deep recurrent q-learning for partially observable mdps. In *AAAI Fall Symposia*, pages 29–37. AAAI Press, 2015.

[125] Rodrigo Toro Icarte, Toryn Q. Klassen, Richard Anthony Valenzano, and Sheila A. McIlraith. Using reward machines for high-level task specification and decomposition in reinforcement learning. In *ICML*, volume 80 of *Proceedings of Machine Learning Research*, pages 2112–2121. PMLR, 2018.

[126] Jan Corazza, Ivan Gavran, and Daniel Neider. Reinforcement learning with stochastic reward machines. In *AAAI*, pages 6429–6436. AAAI Press, 2022.

[127] Fahiem Bacchus, Craig Boutilier, and Adam Grove. Rewarding behaviors. In *Proceedings of the National Conference on Artificial Intelligence*, pages 1160–1167, 1996.

[128] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, 1997.

[129] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*. OpenReview.net, 2021.

[130] Simon Haykin. *Neural networks and learning machines, 3/E*. Pearson Education India, 2009.

[131] Ian Goodfellow, Yoshua Bengio, Aaron Courville, and Yoshua Bengio. *Deep learning*, volume 1. MIT Press, 2016.

[132] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *NeurIPS*, 2023.

[133] Zhiyu Wu, Xiaokang Chen, Zizheng Pan, Xingchao Liu, Wen Liu, Damai Dai, Huazuo Gao, Yiyang Ma, Chengyue Wu, Bingxuan Wang, Zhenda Xie, Yu Wu, Kai Hu, Jiawei Wang, Yaofeng Sun, Yukun Li, Yishi Piao, Kang Guan, Aixin Liu,

Xin Xie, Yuxiang You, Kai Dong, Xingkai Yu, Haowei Zhang, Liang Zhao, Yisong Wang, and Chong Ruan. Deepseek-vl2: Mixture-of-experts vision-language models for advanced multimodal understanding, 2024. URL https://arxiv.org/abs/2412.10302.

[134] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution, 2024. URL https://arxiv.org/abs/2409.12191.

[135] Zhangwei Gao, Zhe Chen, Erfei Cui, Yiming Ren, Weiyun Wang, Jinguo Zhu, Hao Tian, Shenglong Ye, Junjun He, Xizhou Zhu, Lewei Lu, Tong Lu, Yu Qiao, Jifeng Dai, and Wenhai Wang. Mini-internvl: a flexible-transfer pocket multi-modal model with 5% parameters and 90% performance. *Vis. Intell.*, 2(1):32, 2024.

[136] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. In *NAACL-HLT (1)*, pages 4171–4186. Association for Computational Linguistics, 2019.

[137] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *ICML*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763. PMLR, 2021.

[138] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *NeurIPS*, 2022.

[139] Yongliang Wu, Xinting Hu, Yuyang Sun, Yizhou Zhou, Wenbo Zhu, Fengyun Rao, Bernt Schiele, and Xu Yang. Number it: Temporal grounding videos like flipping

manga. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, November 2025.

[140] Weifeng Lin, Xinyu Wei, Ruichuan An, Peng Gao, Bocheng Zou, Yulin Luo, Siyuan Huang, Shanghang Zhang, and Hongsheng Li. Draw-and-understand: Leveraging visual prompts to enable MLLMs to comprehend what you want. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=bfa58H1nQ8.

[141] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI*, 2019. URL https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf. Accessed: 2024-11-15.

[142] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016. URL https://arxiv.org/abs/1607.06450.

[143] Alex Henry, Prudhvi Raj Dachapally, Shubham Shantaram Pawar, and Yuxuan Chen. Query-key normalization for transformers. In *EMNLP (Findings)*, volume EMNLP 2020 of *Findings of ACL*, pages 4246–4253. Association for Computational Linguistics, 2020.

[144] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized BERT pretraining approach. *CoRR*, abs/1907.11692, 2019.

[145] John Firth. A synopsis of linguistic theory, 1930-1955. *Studies in linguistic analysis*, pages 10–32, 1957.

[146] Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting reinforce-style optimization for learning from human feedback in llms. In *ACL (1)*, pages 12248–12267. Association for Computational Linguistics, 2024.

[147] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36:53728–53741, 2023.

[148] Teng Xiao, Yige Yuan, Mingxiao Li, Zhengyu Chen, and Vasant G Honavar. On a connection between imitation learning and RLHF. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=2QdsjiNXgj.

[149] Hao Sun. Reinforcement learning in the era of llms: What is essential? what is needed? an rl perspective on rlhf, prompting, and beyond, 2023. URL https://arxiv.org/abs/2310.06147.

[150] Michael Tschannen, Alexey Gritsenko, Xiao Wang, Muhammad Ferjad Naeem, Ibrahim Alabdulmohsin, Nikhil Parthasarathy, Talfan Evans, Lucas Beyer, Ye Xia, Basil Mustafa, Olivier Hénaff, Jeremiah Harmsen, Andreas Steiner, and Xiaohua Zhai. Siglip 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features, 2025. URL https://arxiv.org/abs/2502.14786.

[151] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *ICCV*, pages 9630–9640. IEEE, 2021.

[152] Haobo Yuan, Xiangtai Li, Tao Zhang, Zilong Huang, Shilin Xu, Shunping Ji, Yunhai Tong, Lu Qi, Jiashi Feng, and Ming-Hsuan Yang. Sa2va: Marrying sam2 with llava for dense grounded understanding of images and videos, 2025. URL https://arxiv.org/abs/2501.04001.

[153] Jingyi Zhang, Jiaxing Huang, Sheng Jin, and Shijian Lu. Vision-language models for vision tasks: A survey, 2024. URL https://arxiv.org/abs/2304.00685.

[154] Muhammad Awais, Muzammal Naseer, Salman Khan, Rao Muhammad Anwer, Hisham Cholakkal, Mubarak Shah, Ming-Hsuan Yang, and Fahad Shahbaz Khan. Foundation models defining a new era in vision: a survey and outlook. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.

[155] Tao Zhang, Xiangtai Li, Hao Fei, Haobo Yuan, Shengqiong Wu, Shunping Ji, Chen Change Loy, and Shuicheng Yan. Omg-llava: Bridging image-level, object-level, pixel-level reasoning and understanding. In *NeurIPS*, 2024.

[156] Yide Shentu, Philipp Wu, Aravind Rajeswaran, and Pieter Abbeel. From llms to actions: Latent codes as bridges in hierarchical robot control. In *IROS*, pages 8539–8546. IEEE, 2024.

[157] Jianke Zhang, Yanjiang Guo, Xiaoyu Chen, Yen-Jen Wang, Yucheng Hu, Chengming Shi, and Jianyu Chen. Hirt: Enhancing robotic control with hierarchical robot transformers. In *CoRL*, volume 270 of *Proceedings of Machine Learning Research*, pages 933–946. PMLR, 2024.

[158] Xinghang Li, Minghuan Liu, Hanbo Zhang, Cunjun Yu, Jie Xu, Hongtao Wu, Chilam Cheang, Ya Jing, Weinan Zhang, Huaping Liu, Hang Li, and Tao Kong. Vision-language foundation models as effective robot imitators. In *ICLR*. OpenReview.net, 2024.

[159] Zhongyi Zhou, Yichen Zhu, Minjie Zhu, Junjie Wen, Ning Liu, Zhiyuan Xu, Weibin Meng, Ran Cheng, Yaxin Peng, Chaomin Shen, and Feifei Feng. Chatvla: Unified multimodal understanding and robot control with vision-language-action model. *CoRR*, abs/2502.14420, 2025.

[160] Kolby Nottingham, Prithviraj Ammanabrolu, Alane Suhr, Yejin Choi, Hannaneh Hajishirzi, Sameer Singh, and Roy Fox. Do embodied agents dream of pixelated sheep: Embodied decision making using language guided world modelling. In *ICML*, volume 202 of *Proceedings of Machine Learning Research*, pages 26311–26325. PMLR, 2023.

[161] Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. Palm-e: An embodied multimodal language model. In *ICML*, volume 202 of *Proceedings of Machine Learning Research*, pages 8469–8488. PMLR, 2023.

[162] Felix Hill, Sona Mokra, Nathaniel Wong, and Tim Harley. Human instruction-following with deep reinforcement learning via transfer-learning from text, 2020. URL https://arxiv.org/abs/2005.09382.

[163] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew J. Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. In *ICLR*. OpenReview.net, 2021.

[164] Weihao Tan, Wentao Zhang, Shanqi Liu, Longtao Zheng, Xinrun Wang, and Bo An. True knowledge comes from practice: Aligning large language models with embodied environments via reinforcement learning. In *ICLR*. OpenReview.net, 2024.

[165] Runlong Zhou, Simon S. Du, and Beibin Li. Reflect-rl: Two-player online RL fine-tuning for lms. In *ACL (1)*, pages 995–1015. Association for Computational Linguistics, 2024.

[166] Jing-Cheng Pang, Xin-Yu Yang, Si-Hang Yang, and Yang Yu. Natural language-conditioned reinforcement learning with inside-out task language development and translation, 2023. URL https://arxiv.org/abs/2302.09368.

[167] Xin Wang, Qiuyuan Huang, Asli Celikyilmaz, Jianfeng Gao, Dinghan Shen, Yuan-Fang Wang, William Yang Wang, and Lei Zhang. Reinforced cross-modal matching and self-supervised imitation learning for vision-language navigation. In *CVPR*, pages 6629–6638. Computer Vision Foundation / IEEE, 2019.

[168] Andrea Frome, Gregory S. Corrado, Jonathon Shlens, Samy Bengio, Jeffrey Dean, Marc'Aurelio Ranzato, and Tomás Mikolov. Devise: A deep visual-semantic embedding model. In *NIPS*, pages 2121–2129, 2013.

[169] Parsa Mahmoudieh, Deepak Pathak, and Trevor Darrell. Zero-shot reward specification via grounded natural language. In *ICML*, volume 162 of *Proceedings of Machine Learning Research*, pages 14743–14752. PMLR, 2022.

[170] Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym, 2016. URL https://arxiv.org/abs/1606.01540.

[171] Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ international conference on intelligent robots and systems*, pages 5026–5033. IEEE, 2012.

[172] Xingyu Lin, Yufei Wang, Jake Olkin, and David Held. Softgym: Benchmarking deep reinforcement learning for deformable object manipulation. In *Conference on Robot Learning*, 2020.

[173] Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *CoRL*, volume 100 of *Proceedings of Machine Learning Research*, pages 1094–1100. PMLR, 2019.

[174] Richard Fikes and Nils J. Nilsson. STRIPS: A new approach to the application of theorem proving to problem solving. In *IJCAI*, pages 608–620. William Kaufmann, 1971.

[175] Constructions Aeronautiques, Adele Howe, Craig Knoblock, ISI Drew McDermott, Ashwin Ram, Manuela Veloso, Daniel Weld, David Wilkins Sri, Anthony Barrett, Dave Christianson, et al. Pddl— the planning domain definition language. 1998.

[176] Drew V. McDermott. The 1998 AI planning systems competition. *AI Mag.*, 21 (2):35–55, 2000.

[177] Maria Fox and Derek Long. PDDL2.1: an extension to PDDL for expressing temporal planning domains. *J. Artif. Intell. Res.*, 20:61–124, 2003.

[178] Alfonso Gerevini, Patrik Haslum, Derek Long, Alessandro Saetti, and Yannis Dimopoulos. Deterministic planning in the fifth international planning competition: PDDL3 and experimental evaluation of the planners. *Artif. Intell.*, 173(5-6):619–668, 2009.

[179] Marcello Balduccini, Daniele Magazzeni, and Marco Maratea. Pddl+ planning via constraint answer set programming, 2016. URL https://arxiv.org/abs/1609.00030.

[180] Peter van Beek and Xinguang Chen. Cplan: A constraint programming approach to planning. In *AAAI/IAAI*, pages 585–590. AAAI Press / The MIT Press, 1999.

[181] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans. Syst. Sci. Cybern.*, 4(2):100–107, 1968.

[182] Judea Pearl. *Heuristics: intelligent search strategies for computer problem solving.* Addison-Wesley Longman Publishing Co., Inc., 1984.

[183] Jörg Hoffmann and Bernhard Nebel. The FF planning system: Fast plan generation through heuristic search. *J. Artif. Intell. Res.*, 14:253–302, 2001.

[184] Blai Bonet and Hector Geffner. Planning as heuristic search: New results. In *ECP*, volume 1809 of *Lecture Notes in Computer Science*, pages 360–372. Springer, 1999.

[185] Stefan Edelkamp. Symbolic pattern databases in heuristic search planning. In *AIPS*, pages 274–283. AAAI, 2002.

[186] Malte Helmert, Patrik Haslum, and Jörg Hoffmann. Flexible abstraction heuristics for optimal sequential planning. In *ICAPS*, pages 176–183. AAAI, 2007.

[187] Raz Nissim, Jörg Hoffmann, and Malte Helmert. Computing perfect heuristics in polynomial time: On bisimulation and merge-and-shrink abstraction in optimal planning. In *IJCAI*, pages 1983–1990. IJCAI/AAAI, 2011.

[188] Malte Helmert and Carmel Domshlak. Landmarks, critical paths and abstractions: What's the difference anyway? In *ICAPS*. AAAI, 2009.

[189] J. Scott Penberthy and Daniel S. Weld. UCPOP: A sound, complete, partial order planner for ADL. In *KR*, pages 103–114. Morgan Kaufmann, 1992.

[190] Nir Lipovetzky and Hector Geffner. Width and serialization of classical planning problems. In *ECAI*, volume 242 of *Frontiers in Artificial Intelligence and Applications*, pages 540–545. IOS Press, 2012.

[191] Nir Lipovetzky and Hector Geffner. Best-first width search: Exploration and exploitation in classical planning. In *AAAI*, pages 3590–3596. AAAI Press, 2017.

[192] Kulin Shah, Nishanth Dikkala, Xin Wang, and Rina Panigrahy. Causal language modeling can elicit search and reasoning capabilities on logic puzzles. In *NeurIPS*, 2024.

[193] Guoxin Chen, Minpeng Liao, Chengxi Li, and Kai Fan. Alphamath almost zero: Process supervision without process. In *NeurIPS*, 2024.

[194] Yongchao Chen, Rujul Gandhi, Yang Zhang, and Chuchu Fan. NL2TL: transforming natural languages to temporal logics using large language models. In *EMNLP*, pages 15880–15903. Association for Computational Linguistics, 2023.

[195] Yongchao Chen, Jacob Arkin, Charles Dawson, Yang Zhang, Nicholas Roy, and Chuchu Fan. Autotamp: Autoregressive task and motion planning with llms as translators and checkers. In *ICRA*, pages 6695–6702. IEEE, 2024.

[196] Zhirui Dai, Arash Asgharivaskasi, Thai Duong, Shusen Lin, Maria-Elizabeth Tzes, George J. Pappas, and Nikolay Atanasov. Optimal scene graph planning with large language model guidance. In *ICRA*, pages 14062–14069. IEEE, 2024.

[197] Michael Katz, Harsha Kokel, Kavitha Srinivas, and Shirin Sohrabi. Thought of search: Planning with language models through the lens of efficiency. In *NeurIPS*, 2024.

[198] Daniel Cao, Michael Katz, Harsha Kokel, Kavitha Srinivas, and Shirin Sohrabi. Automating thought of search: A journey towards soundness and completeness, 2024. URL https://arxiv.org/abs/2408.11326.

[199] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

[200] Murtaza Dalal, Tarun Chiruvolu, Devendra Singh Chaplot, and Ruslan Salakhutdinov. Plan-seq-learn: Language model guided RL for solving long horizon robotics tasks. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL https://openreview.net/forum?id=hQVCCxQrYN.

[201] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In Anna Rogers, Jordan L. Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023,*

Toronto, Canada, July 9-14, 2023, pages 2609–2634. Association for Computational Linguistics, 2023. doi: 10.18653/V1/2023.ACL-LONG.147. URL https://doi.org/10.18653/v1/2023.acl-long.147.

[202] Daniel Kahneman. *Thinking, fast and slow*. macmillan, 2011.

[203] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. Graph of thoughts: Solving elaborate problems with large language models. In Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan, editors, *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2014, February 20-27, 2024, Vancouver, Canada*, pages 17682–17690. AAAI Press, 2024. doi: 10.1609/AAAI.V38I16.29720. URL https://doi.org/10.1609/aaai.v38i16.29720.

[204] OpenAI, :, Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, et al. Openai o1 system card, 2024. URL https://arxiv.org/abs/2412.16720.

[205] Kaya Stechly, Karthik Valmeekam, and Subbarao Kambhampati. Chain of thoughtlessness? an analysis of cot in planning. In *NeurIPS*, 2024.

[206] Karthik Valmeekam, Kaya Stechly, Atharva Gundawar, and Subbarao Kambhampati. Planning in strawberry fields: Evaluating and improving the planning and scheduling capabilities of lrm o1, 2024. URL https://arxiv.org/abs/2410.02162.

[207] Danijar Hafner. Benchmarking the spectrum of agent capabilities. In *ICLR*. OpenReview.net, 2022.

[208] Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, Rodrigo Perez-Vicente, Lucas Willems, Salem Lahlou, Suman Pal, Pablo Samuel Castro, and Jordan K. Terry. Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. In *NeurIPS*, 2023.

[209] Yue Wu, So Yeon Min, Shrimai Prabhumoye, Yonatan Bisk, Russ R Salakhutdinov, Amos Azaria, Tom M Mitchell, and Yuanzhi Li. Spring: Studying papers and

reasoning to play games. *Advances in Neural Information Processing Systems*, 36: 22383–22687, 2023.

[210] Seungyong Moon, Junyoung Yeom, Bumsoo Park, and Hyun Oh Song. Discovering hierarchical achievements in reinforcement learning via contrastive learning. *Advances in Neural Information Processing Systems*, 36:63674–63686, 2023.

[211] Jendrik Seipp, Álvaro Torralba, and Jörg Hoffmann. PDDL generators. https://doi.org/10.5281/zenodo.6382173, 2022.

[212] Mohit Shridhar et al. Cliport: What and where pathways for robotic manipulation. In *Conference on robot learning*, pages 894–906. PMLR, 2022.

[213] Masatoshi Uehara and Wen Sun. Pessimistic model-based offline reinforcement learning under partial coverage. *arXiv preprint arXiv:2107.06226*, 2021.

[214] Chenjia Bai et al. Pessimistic value iteration for multi-task data sharing in offline reinforcement learning. *Artificial Intelligence*, 326:104048, 2024.

[215] Gaurav R. Ghosal, Matthew Zurek, Daniel S. Brown, and Anca D. Dragan. The effect of modeling human rationality level on learning rewards from multiple feedback types. In *AAAI Conference on Artificial Intelligence*, 2022.

[216] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline reinforcement learning. *Advances in Neural Information Processing Systems*, 33:1179–1191, 2020.

[217] Ying Jin, Zhuoran Yang, and Zhaoran Wang. Is pessimism provably efficient for offline rl? In *International Conference on Machine Learning*, pages 5084–5096. PMLR, 2021.

[218] Nouha Dziri et al. Faith and fate: Limits of transformers on compositionality. *Advances in Neural Information Processing Systems*, 36, 2024.

[219] Thang M Pham, Trung Bui, Long Mai, and Anh Nguyen. Out of order: How important is the sequential order of words in a sentence in natural language understanding tasks? *arXiv preprint arXiv:2012.15180*, 2020.

[220] Maxime Chevalier-Boisvert, Dzmitry Bahdanau, Salem Lahlou, Lucas Willems, Chitwan Saharia, Thien Huu Nguyen, and Yoshua Bengio. Babyai: A platform

to study the sample efficiency of grounded language learning. In *International Conference on Learning Representations*, 2018.

[221] Yiwei Ma, Guohai Xu, Xiaoshuai Sun, Ming Yan, Ji Zhang, and Rongrong Ji. X-clip: End-to-end multi-grained contrastive learning for video-text retrieval. *Proceedings of the 30th ACM International Conference on Multimedia*, 2022.

[222] Md Mosharaf Hossain et al. An analysis of negation in natural language understanding corpora. *arXiv preprint arXiv:2203.08929*, 2022.

[223] Thinh Hung Truong, Timothy Baldwin, Karin Verspoor, and Trevor Cohn. Language models are not naysayers: an analysis of language models on negation benchmarks. *arXiv preprint arXiv:2306.08189*, 2023.

[224] Mauricio Sadinle et al. Least ambiguous set-valued classifiers with bounded error levels. *Journal of the American Statistical Association*, 114(525):223–234, 2019.

[225] Mengdi Li, Xufeng Zhao, Jae Hee Lee, Cornelius Weber, and Stefan Wermter. Internally rewarded reinforcement learning. In *International Conference on Machine Learning*, pages 20556–20574. PMLR, 2023.

[226] Xiao Ma, Bingyi Kang, Zhongwen Xu, Min Lin, and Shuicheng Yan. Mutual information regularized offline reinforcement learning. In *NeurIPS*, 2023.

[227] Matthew Hausknecht and Peter Stone. Deep recurrent q-learning for partially observable mdps. In *2015 aaai fall symposium series*, 2015.

[228] Krishan Rana, Jesse Haviland, Sourav Garg, Jad Abou-Chakra, Ian Reid, and Niko Suenderhauf. Sayplan: Grounding large language models using 3d scene graphs for scalable robot task planning. In *7th Annual Conference on Robot Learning*, 2023.

[229] James Oswald, Kavitha Srinivas, Harsha Kokel, Junkyu Lee, Michael Katz, and Shirin Sohrabi. Large language models as planning domain generators. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 34, pages 423–431, 2024.

[230] Tom Silver, Soham Dan, Kavitha Srinivas, Joshua B. Tenenbaum, Leslie Pack Kaelbling, and Michael Katz. Generalized planning in pddl domains with pretrained large language models. In *AAAI Conference on Artificial Intelligence*, 2023. URL https://api.semanticscholar.org/CorpusID:258762760.

[231] Julius M Moravcsik. Natural languages and formal languages: a tenable dualism. In *Language, Logic and Method*, pages 225–239. Springer, 1983.

[232] Jiannan Xiang, Tianhua Tao, Yi Gu, Tianmin Shu, Zirui Wang, Zichao Yang, and Zhiting Hu. Language models meet world models: Embodied experiences enhance language models. *Advances in neural information processing systems*, 36, 2024.

[233] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *International conference on machine learning*, pages 9118–9147. PMLR, 2022.

[234] Stephanie C. Lin et al. Teaching models to express their uncertainty in words. *Trans. Mach. Learn. Res.*, 2022, 2022. URL https://api.semanticscholar.org/CorpusID:249191391.

[235] Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley, and Wayne Xin Zhao. Large language models are zero-shot rankers for recommender systems. In *European Conference on Information Retrieval*, 2023. URL https://api.semanticscholar.org/CorpusID:258686540.

[236] Shengyao Zhuang, Bing Liu, Bevan Koopman, and Guido Zuccon. Open-source large language models are strong zero-shot query likelihood models for document ranking. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 8807–8817, 2023.

[237] Steven A. Sloman. The empirical case for two systems of reasoning. *Psychological Bulletin*, 119:3–22, 1996. URL https://api.semanticscholar.org/CorpusID:13454019.

[238] Kira Mourao et al. Using kernel perceptrons to learn action effects for planning. In *Proceedings of the International Conference on Cognitive Systems (CogSys 2008)*, pages 45–50, 2008.

[239] Diego Aineto et al. Learning action models with minimal observability. *Artificial Intelligence*, 275:104–137, 2019.

[240] Constructions Aeronautiques, Adele Howe, Craig Knoblock, ISI Drew McDermott, Ashwin Ram, Manuela Veloso, Daniel Weld, David Wilkins Sri, Anthony Barrett, Dave Christianson, et al. Pddl— the planning domain definition language. *Technical Report, Tech. Rep.*, 1998.

[241] Warren Weaver. Translation. In *Proceedings of the Conference on Mechanical Translation*, 1952.

[242] Mark Chen, Jerry Tworek, et al. Evaluating large language models trained on code, 2021. URL https://arxiv.org/abs/2107.03374.

[243] Baptiste Rozière, Jonas Gehring, et al. Code llama: Open foundation models for code, 2024. URL https://arxiv.org/abs/2308.12950.

[244] Joshua David Robinson, Ching-Yao Chuang, Suvrit Sra, and Stefanie Jegelka. Contrastive learning with hard negative samples. In *International Conference on Learning Representations*, 2023.

[245] Malte Helmert. Concise finite-domain representations for pddl planning tasks. *Artificial Intelligence*, 173(5-6):503–535, 2009.

[246] Lukáš Chrpa et al. The fifth international competition on knowledge engineering for planning and scheduling: Summary and trends. *AI Magazine*, 38(1):104–106, 2017.

[247] Tom Silver and Rohan Chitnis. Pddlgym: Gym environments from pddl problems. In *ICAPS Workshop on Bridging the Gap Between AI Planning and Reinforcement Learning (PRL)*, 2020.

[248] Zhenyu Hou, Yilin Niu, Zhengxiao Du, Xiaohan Zhang, Xiao Liu, Aohan Zeng, Qinkai Zheng, Minlie Huang, Hongning Wang, Jie Tang, and Yuxiao Dong. Chatglm-rlhf: Practices of aligning large language models with human feedback, 2024. URL https://arxiv.org/abs/2404.00934.

[249] Gautier Dagan, Frank Keller, and Alex Lascarides. Dynamic planning with a LLM. *CoRR*, abs/2308.06391, 2023.

[250] Jiyoun Moon and Beom-Hee Lee. Pddl planning with natural language-based scene understanding for uav-ugv cooperation. *Applied Sciences*, 9(18), 2019. ISSN 2076-3417. doi: 10.3390/app9183789. URL https://www.mdpi.com/2076-3417/9/18/3789.

[251] Yaqi Xie, Chen Yu, Tongyao Zhu, Jinbin Bai, Ze Gong, and Harold Soh. Translating natural language to planning goals with large-language models. *CoRR*, abs/2302.05128, 2023.

[252] Alex Coulter, Teo Ilie, Renee Tibando, and Christian Muise. Theory alignment via a classical encoding of regular bisimulation. In *Proceedings of the Workshop on Knowledge Engineering for Planning and Scheduling (KEPS) at ICAPS*. ICAPS, 2022.

[253] OpenAI, :, Aaron Hurst, Adam Lerer, Adam P. Goucher, et al. Gpt-4o system card, 2024. URL https://arxiv.org/abs/2410.21276.

[254] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.

[255] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*, 2024.

[256] Bingbin Liu, Sebastien Bubeck, Ronen Eldan, Janardhan Kulkarni, Yuanzhi Li, Anh Nguyen, Rachel Ward, and Yi Zhang. Tinygsm: achieving¿ 80% on gsm8k with small language models. *arXiv preprint arXiv:2312.09241*, 2023.

[257] Tian Ye, Zicheng Xu, Yuanzhi Li, and Zeyuan Allen-Zhu. Physics of language models: Part 2.2, how to learn from mistakes on grade-school math problems. *ArXiv*, abs/2408.16293, 2024. URL https://api.semanticscholar.org/CorpusID:272146356.

[258] Aviral Kumar, Vincent Zhuang, Rishabh Agarwal, Yi Su, John D Co-Reyes, Avi Singh, Kate Baumli, Shariq Iqbal, Colton Bishop, Rebecca Roelofs, et al. Training language models to self-correct via reinforcement learning. *arXiv preprint arXiv:2409.12917*, 2024.

[259] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2024.

[260] Iman Mirzadeh, Keivan Alizadeh, Hooman Shahrokhi, Oncel Tuzel, Samy Bengio, and Mehrdad Farajtabar. Gsm-symbolic: Understanding the limitations of mathematical reasoning in large language models. *arXiv preprint arXiv:2410.05229*, 2024.

[261] Zeyuan Allen-Zhu and Yuanzhi Li. Physics of language models: Part 3.1, knowledge storage and extraction. *arXiv preprint arXiv:2309.14316*, 2023.

[262] Ekin D Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V Le. Randaugment: Practical data augmentation with no separate search. *arXiv preprint arXiv:1909.13719*, 2(4):7, 2019.

[263] Raphael Gontijo-Lopes, Sylvia J Smullin, Ekin D Cubuk, and Ethan Dyer. Affinity and diversity: Quantifying mechanisms of data augmentation. *arXiv preprint arXiv:2002.08973*, 2020.

[264] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *ICLR*. OpenReview.net, 2023.

[265] Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang, Suraj Srivats, Soroush Vosoughi, Hyung Won Chung, Yi Tay, Sebastian Ruder, Denny Zhou, Dipanjan Das, and Jason Wei. Language models are multilingual chain-of-thought reasoners. In *ICLR*. OpenReview.net, 2023.

[266] Ben Prystawski, Michael Li, and Noah D. Goodman. Why think step by step? reasoning emerges from the locality of experience. In *NeurIPS*, 2023.

[267] Chenxin An, Jiangtao Feng, Kai Lv, Lingpeng Kong, Xipeng Qiu, and Xuanjing Huang. Cont: Contrastive neural text generation. *Advances in Neural Information Processing Systems*, 35:2197–2210, 2022.

[268] Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O Stanley, and Jeff Clune. Go-explore: a new approach for hard-exploration problems. *arXiv preprint arXiv:1901.10995*, 2019.

[269] Michael Katz, Junkyu Lee, and Shirin Sohrabi. Unifying and certifying top-quality planning. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 34, pages 319–323, 2024.

[270] Stephen V. Chenoweth. On the np-hardness of blocks world. In *AAAI Conference on Artificial Intelligence*, 1991. URL https://api.semanticscholar.org/CorpusID:15078759.

[271] Antoine Gorceix, Bastien Le Chenadec, Ahmad Rammal, Nelson Vadori, and Manuela Veloso. Learning mathematical rules with large language models. In *The 4th Workshop on Mathematical Reasoning and AI at NeurIPS'24*, 2024.

[272] Gan Chen, Yi Ding, Hugo Edwards, Chong Hin Chau, Sai Hou, Grace Johnson, Mohammed Sharukh Syed, Haoyuan Tang, Yue Wu, Ye Yan, Tidhar Gil, and Lipovetzky Nir. Planimation. *arXiv preprint arXiv:2008.04600*, 2020.

[273] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

[274] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *ICML*. OpenReview.net, 2024.

[275] Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Sergey Levine, and Yi Ma. SFT memorizes, RL generalizes: A comparative study of foundation model post-training. In *The Second Conference on Parsimony and Learning (Recent Spotlight Track)*, 2025. URL https://openreview.net/forum?id=d3E3LWmTar.

[276] Yi-Chun Chen, Mykel J Kochenderfer, and Matthijs TJ Spaan. Improving offline value-function approximations for pomdps by reducing discount factors. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3531–3536. IEEE, 2018.

[277] Zhuosheng Zhang, Aston Zhang, Mu Li, and Alex Smola. Automatic chain of thought prompting in large language models. In *ICLR*. OpenReview.net, 2023.